南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY
生物医学工程系
Department of Biomedical Engineering

Spatial Filtering

Yiming Li
liym2019@sustech.edu.cn

2026/10/30

Learning Objectives

- Compute a filter output from a small neighborhood.
- Specify the kernel, boundary rule, and output size.
- Choose smoothing or sharpening for the image task.
- Check noise, detail loss, and processing artifacts.

Lecture Roadmap

- Neighborhood operations and implementation
- Smoothing, noise, and the median filter
- Derivatives and sharpening
- Combining methods and evaluating the result

Same Intensity, Different Contexts

A: constant patch

100	100	100
100	**100**	100
100	100	100

B: isolated bright point

0	0	0
0	**100**	0
0	0	0

Can a point mapping distinguish the center pixels?
What information does a neighborhood add?

What is a Filter?

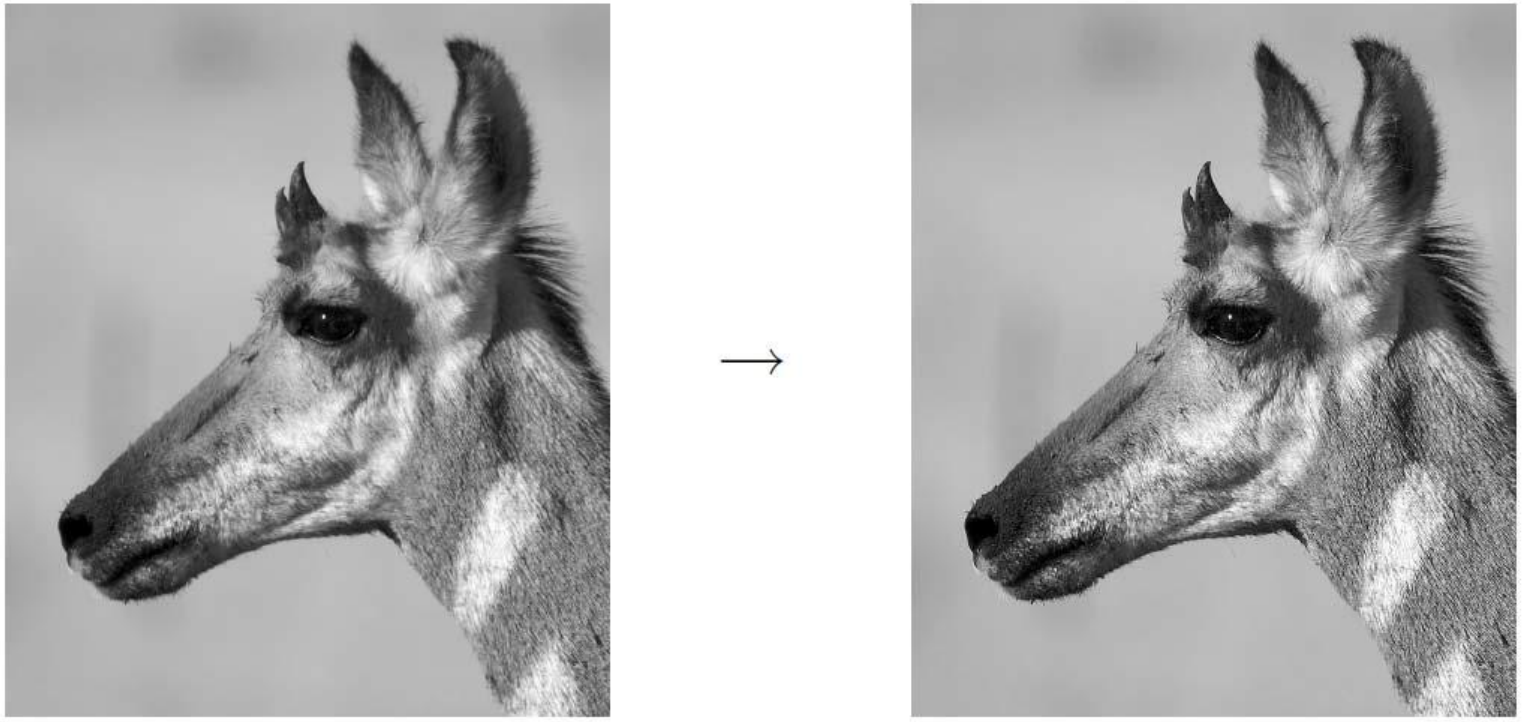
- Capabilities of point operations are limited
- **Filters:** combine **pixel's value + values of neighbours**
- **E.g. blurring:** Compute average intensity of block of pixels



- Combining multiple pixels needed for certain operations:
  - Blurring, Smoothing
  - Sharpening

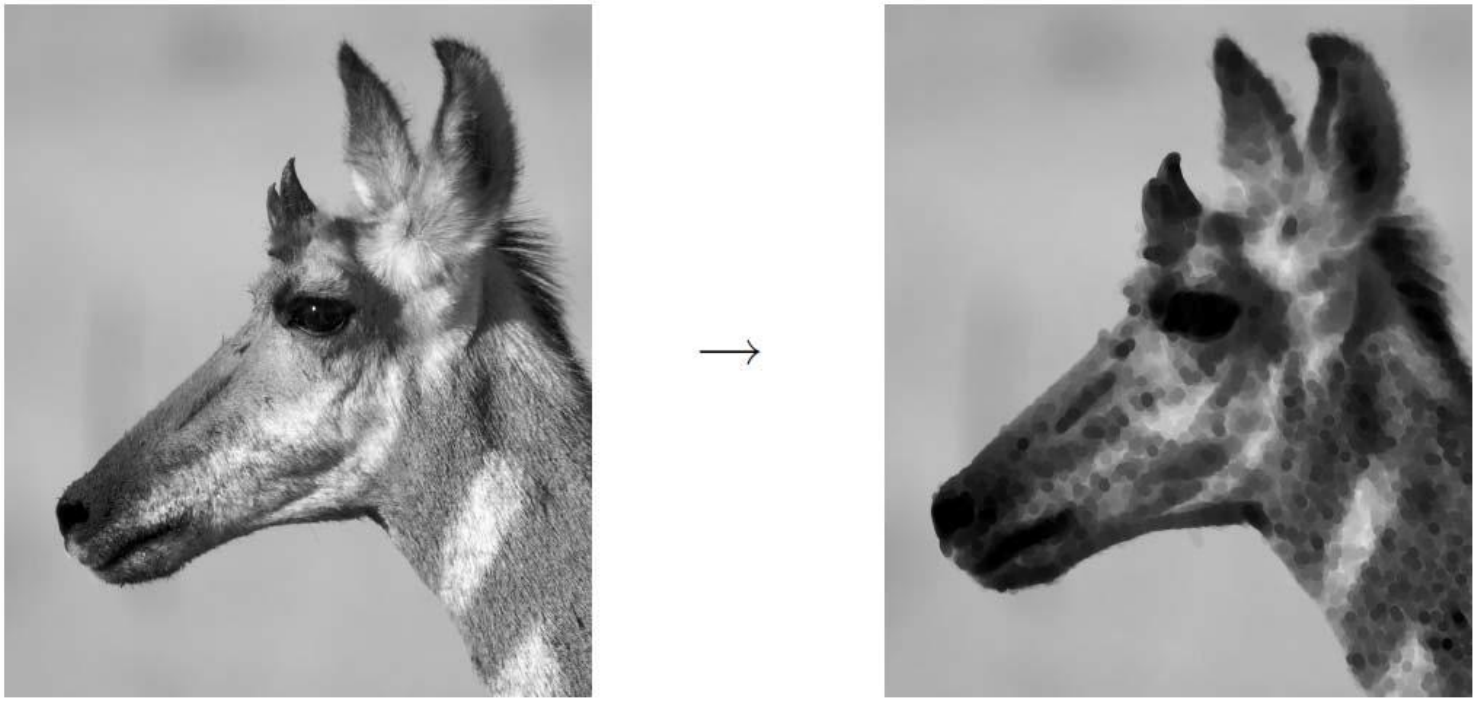
Example: Sharpening

- Example: sharpening



What Point Operations Can't Do

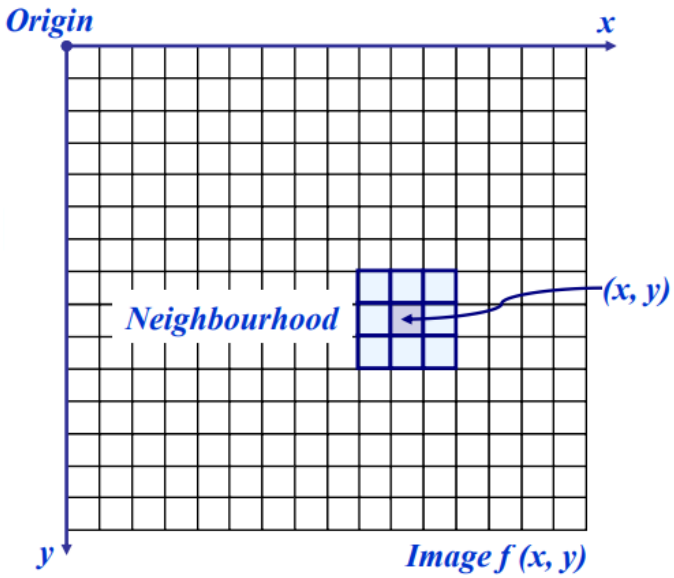
- Other cool artistic patterns by combining pixels



Neighborhood Operations

Neighborhood operations

- Neighborhood operations simply operate on a larger neighborhood of pixels than point operations
- Neighborhoods are mostly a rectangle around a central pixel
- Any size rectangle and any shape filter are possible



Filter Parameters

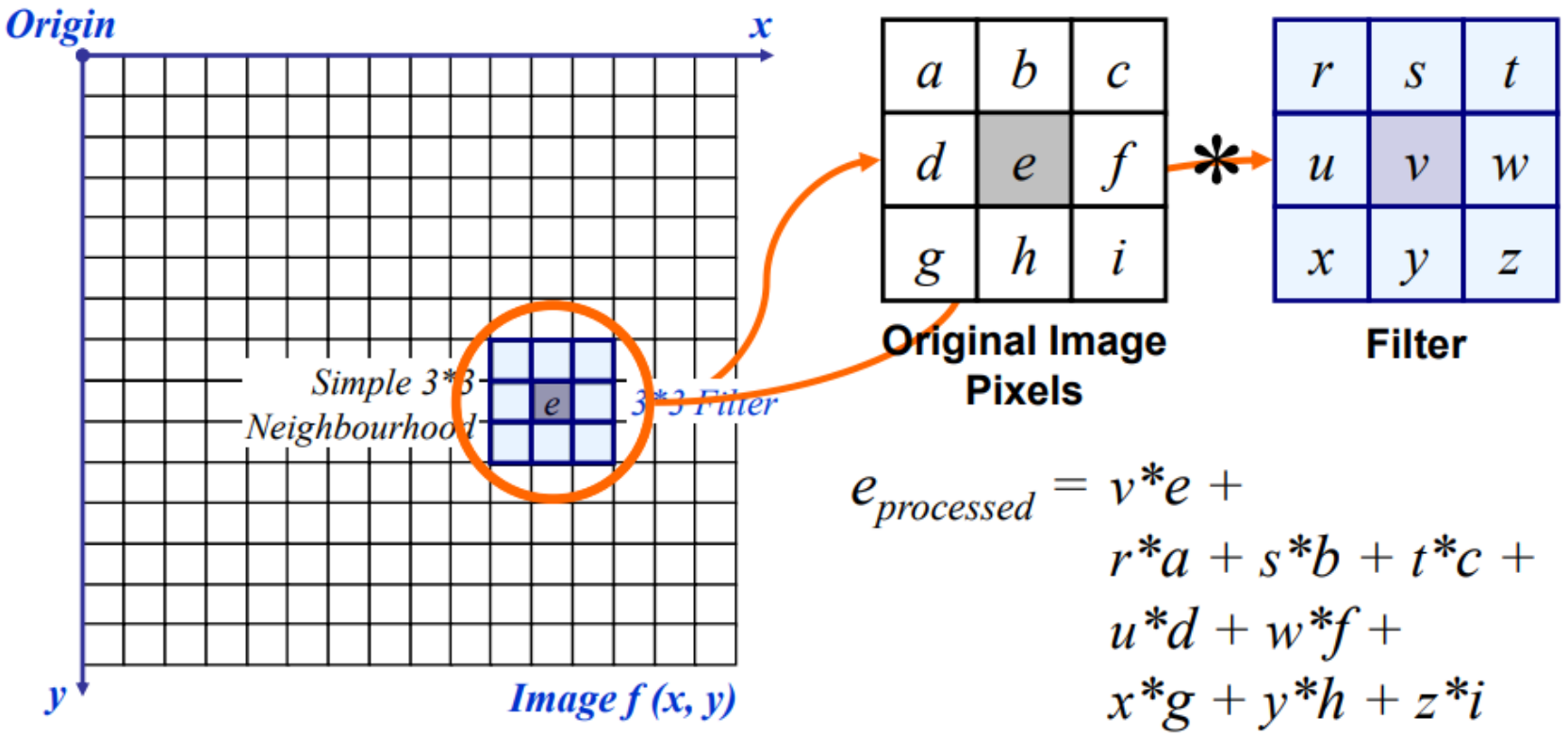
- Many possible filter parameters (size, weights, function, etc)
- **Filter size (size of neighbourhood):**

  3x3, 5x5, 7x7, …,21x21,..
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels
- **Filter function:** can be linear (a weighted summation) or nonlinear

Simple Neighborhood Operations

- Some simple neighborhood operations
  - Min: Set the pixel value to the minimum in the neighborhood
  - Max: Set the pixel value to the maximum in the neighborhood
  - Median: The median value of a set of numbers is the midpoint value in that set (e.g. from the set [1, 7, 15, 18, 24] the median is 15). Sometimes the median works better than the average

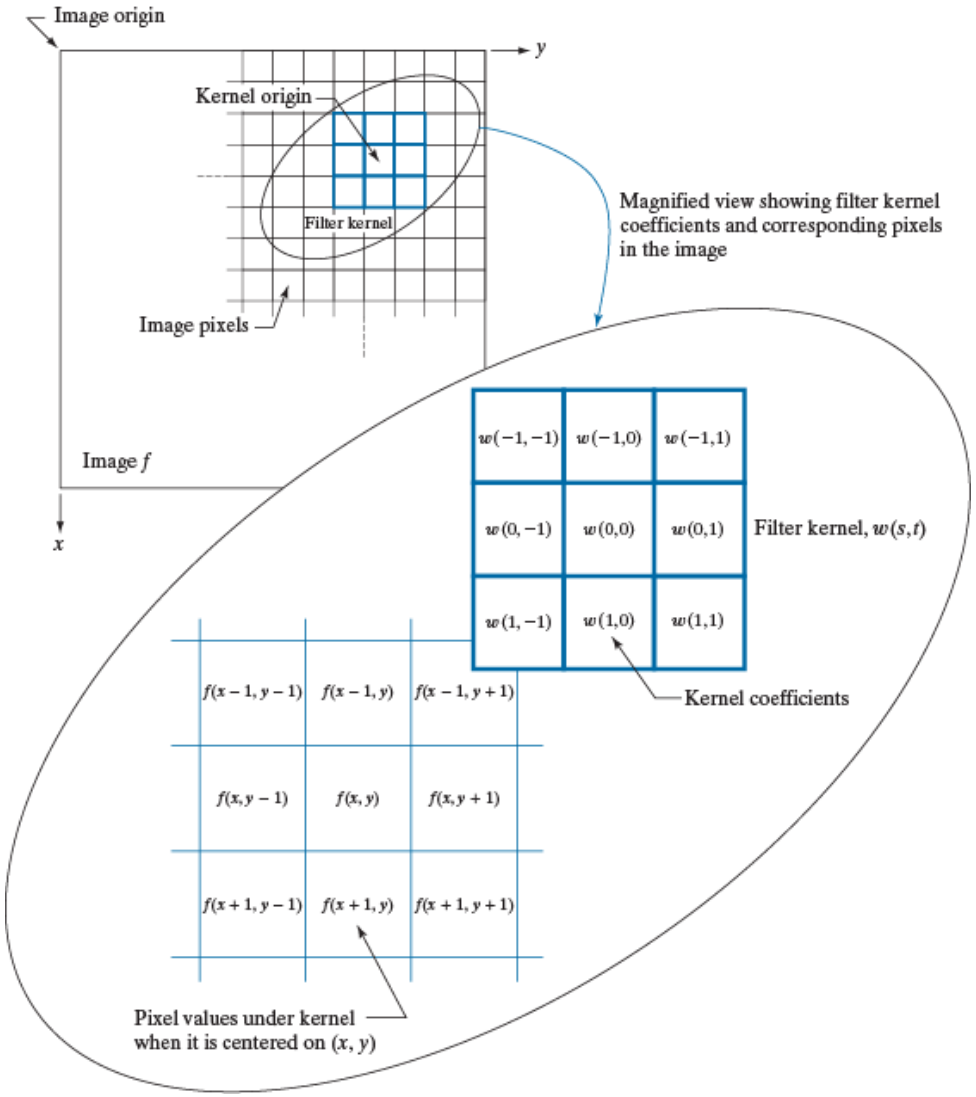
The Spatial Filtering Process



$$e_{processed} = v*e + r*a + s*b + t*c + u*d + w*f + x*g + y*h + z*i$$

The above is repeated for every pixel in the original image to generate the filtered image

Spatial Filtering: Equation Form

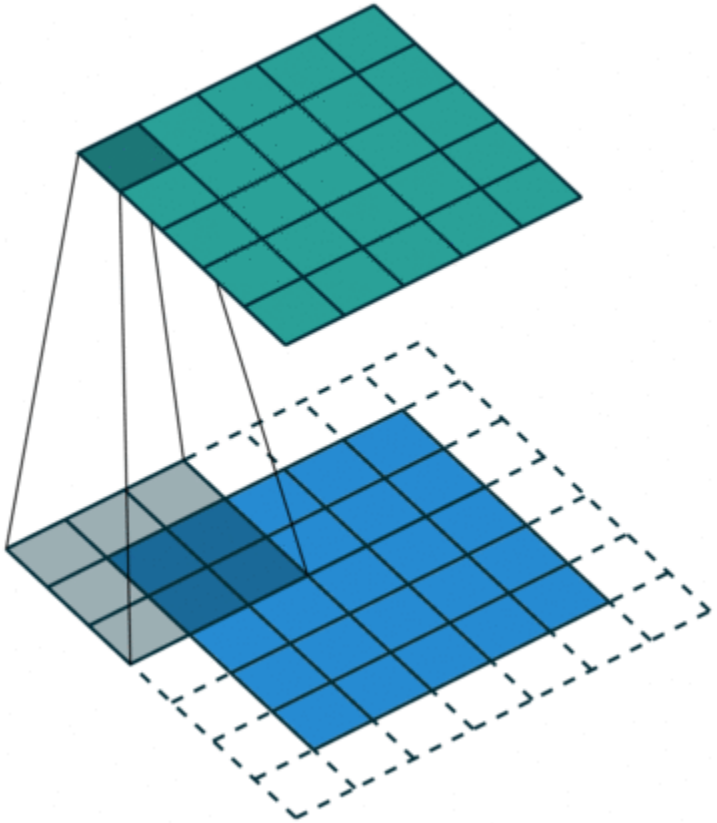


$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

Filtering can be given in equation form as shown above

Notations are based on the image shown to the left

Spatial Filtering: Equation Form

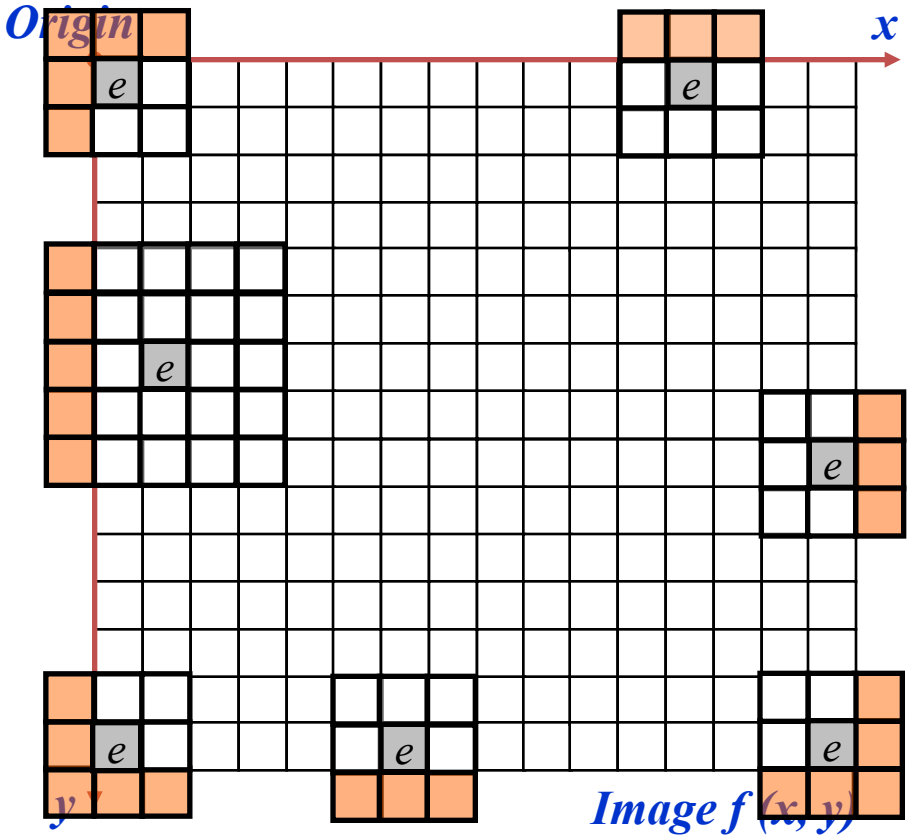


$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

Filtering can be given in equation form as shown above

Notations are based on the image shown to the left

At the edges of an image, we are missing pixels to form a neighbourhood



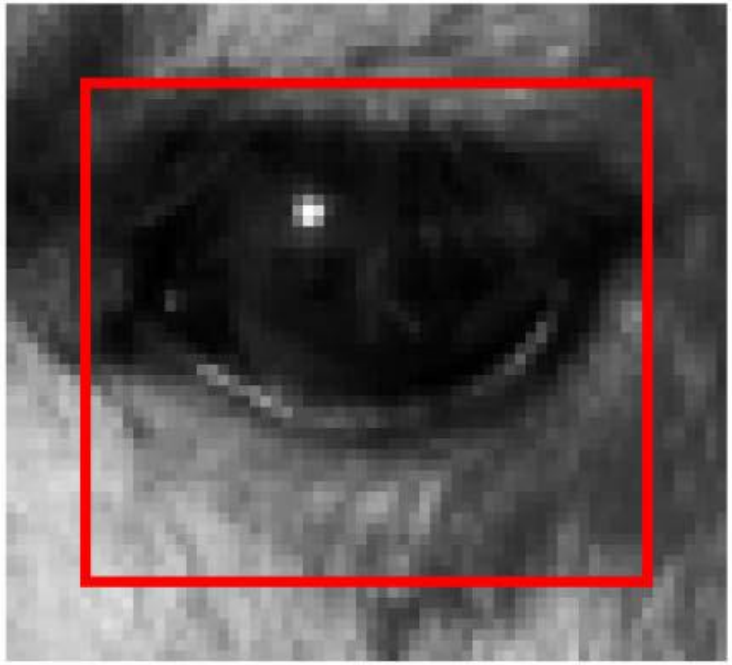


Strange Things Happen At The Edges! (cont…)

- There are a few approaches to dealing with missing edge pixels:
  - Omit missing pixels
    - Only works with some filters
    - Can add extra code and slow down processing
  - Pad the image
    - Typically with either all white or all black pixels
  - Replicate border pixels (Extend)
  - Truncate the image (crop)
  - Allow pixels *wrap around* the image
    - Can cause some strange image artefacts

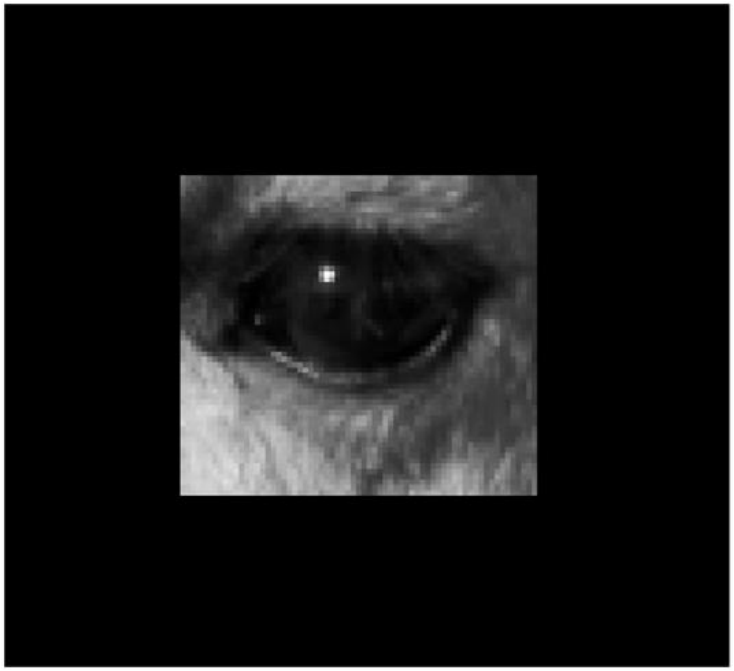
What to do at image boundaries?

Crop



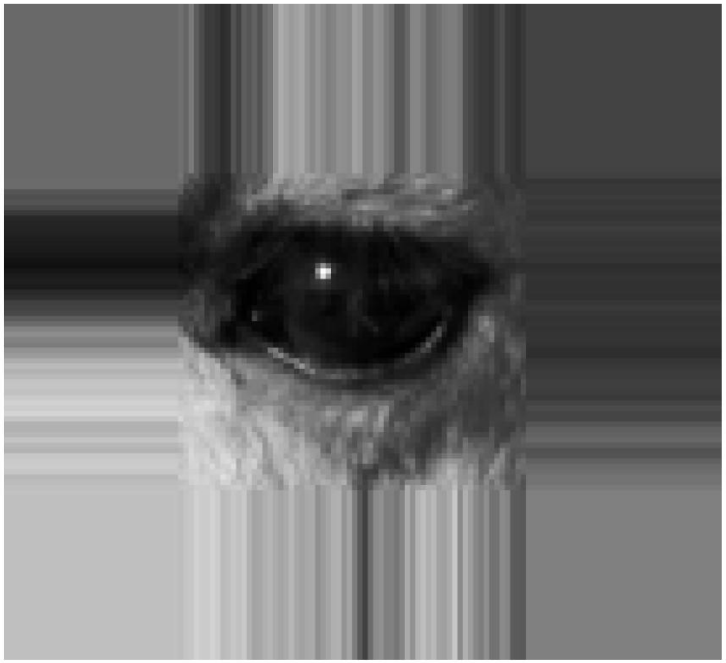
What to do at image boundaries?

Pad



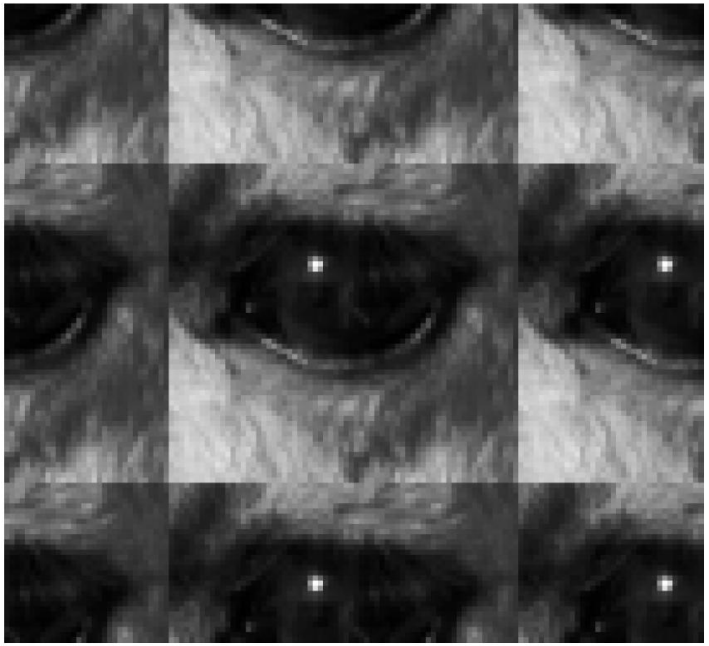
What to do at image boundaries?

Extend

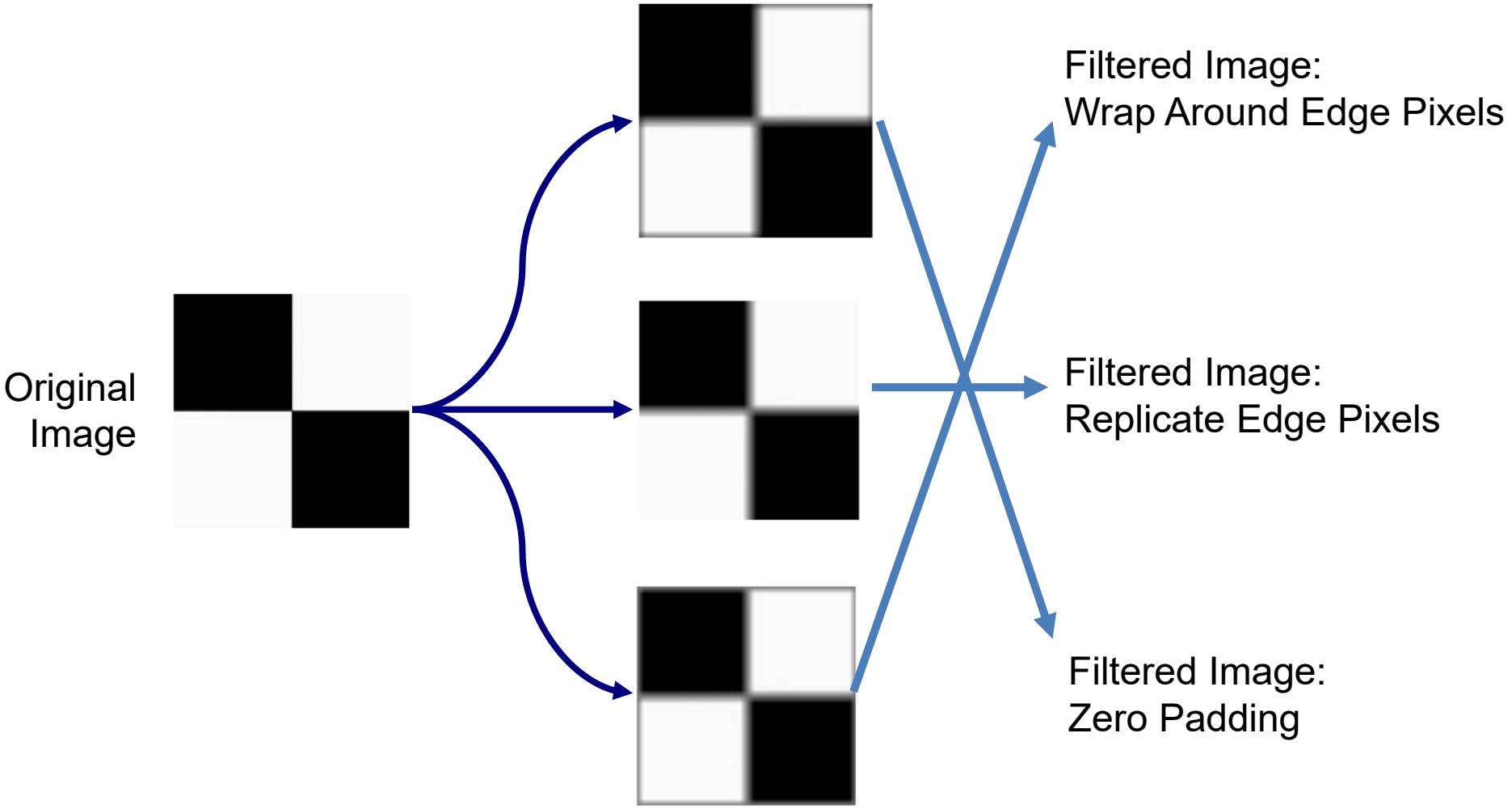


What to do at image boundaries?

Wrap



Strange Things Happen At The Edges! (cont…)



Correlation & Convolution

- The filtering we have been talking about so far is referred to as *correlation* with the filter itself referred to as the *correlation kernel*
- *Convolution* is a similar operation, with just one subtle difference

a	*b*	*c*
d	*e*	*e*
f	*g*	*h*

Original Image Pixels

$*$

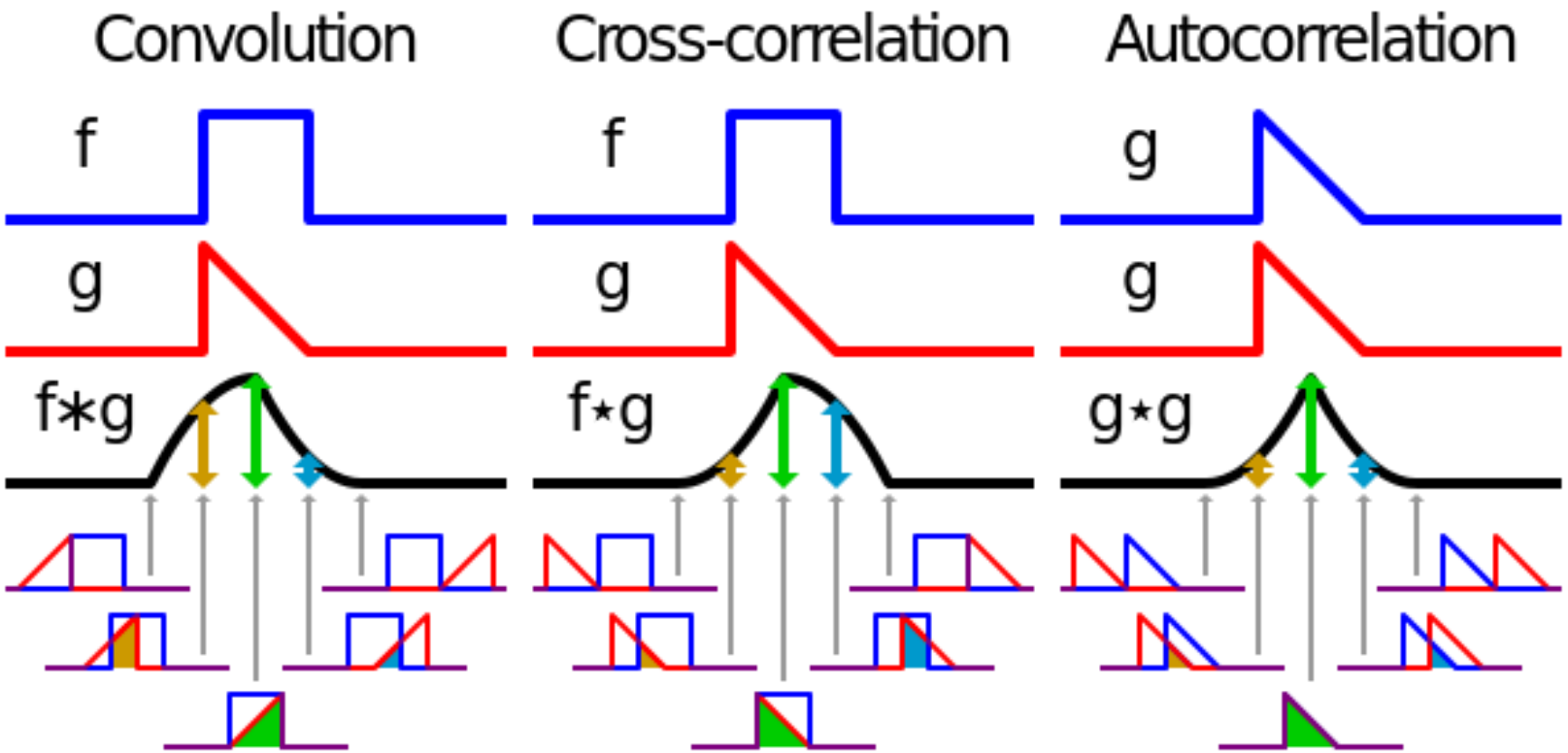
r	*s*	*t*
u	*v*	*w*
x	*y*	*z*

Filter

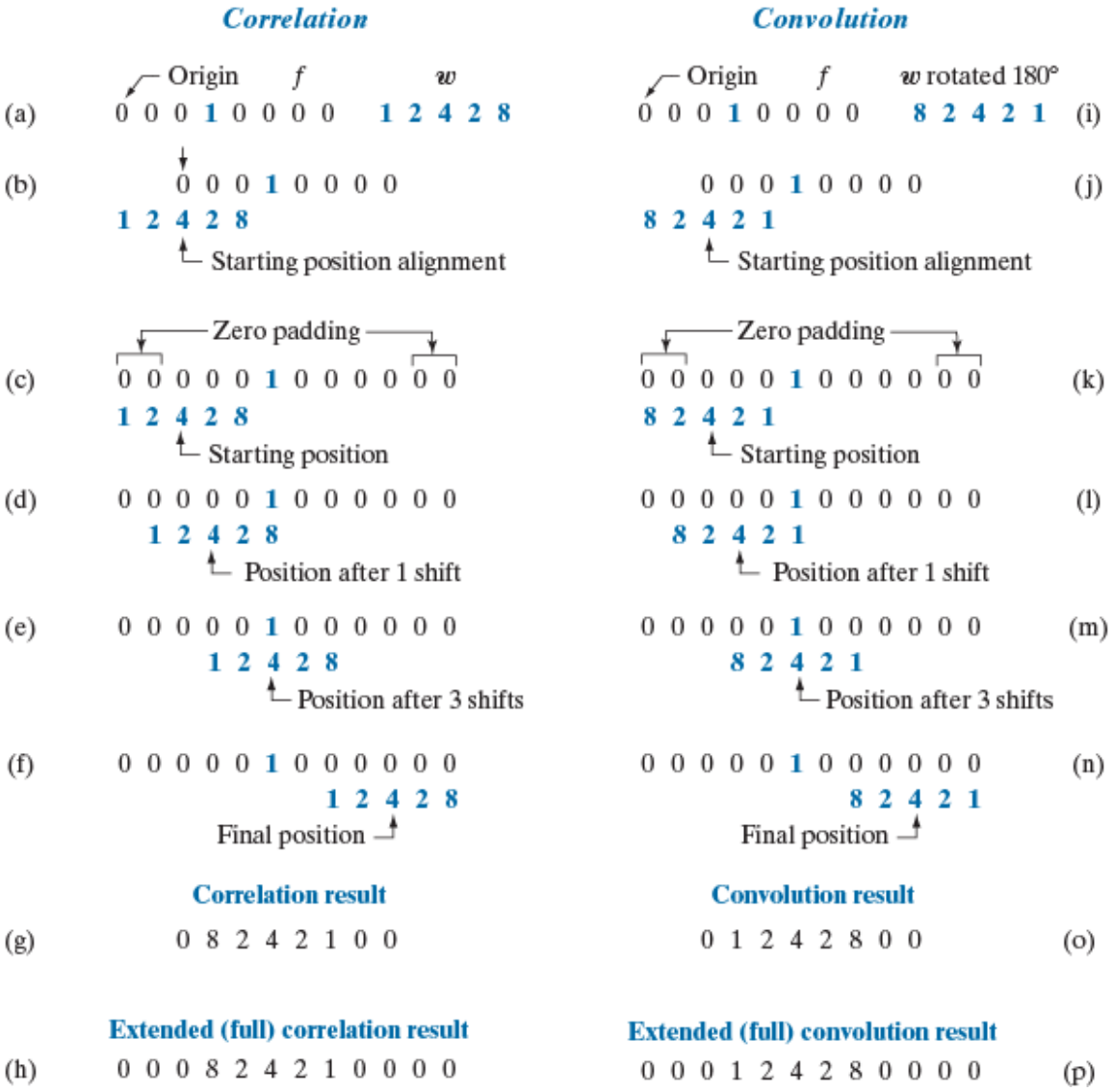
$$e_{processed} = v*e + z*a + y*b + x*c + w*d + u*e + t*f + s*g + r*h$$

- For symmetric filters it makes no difference

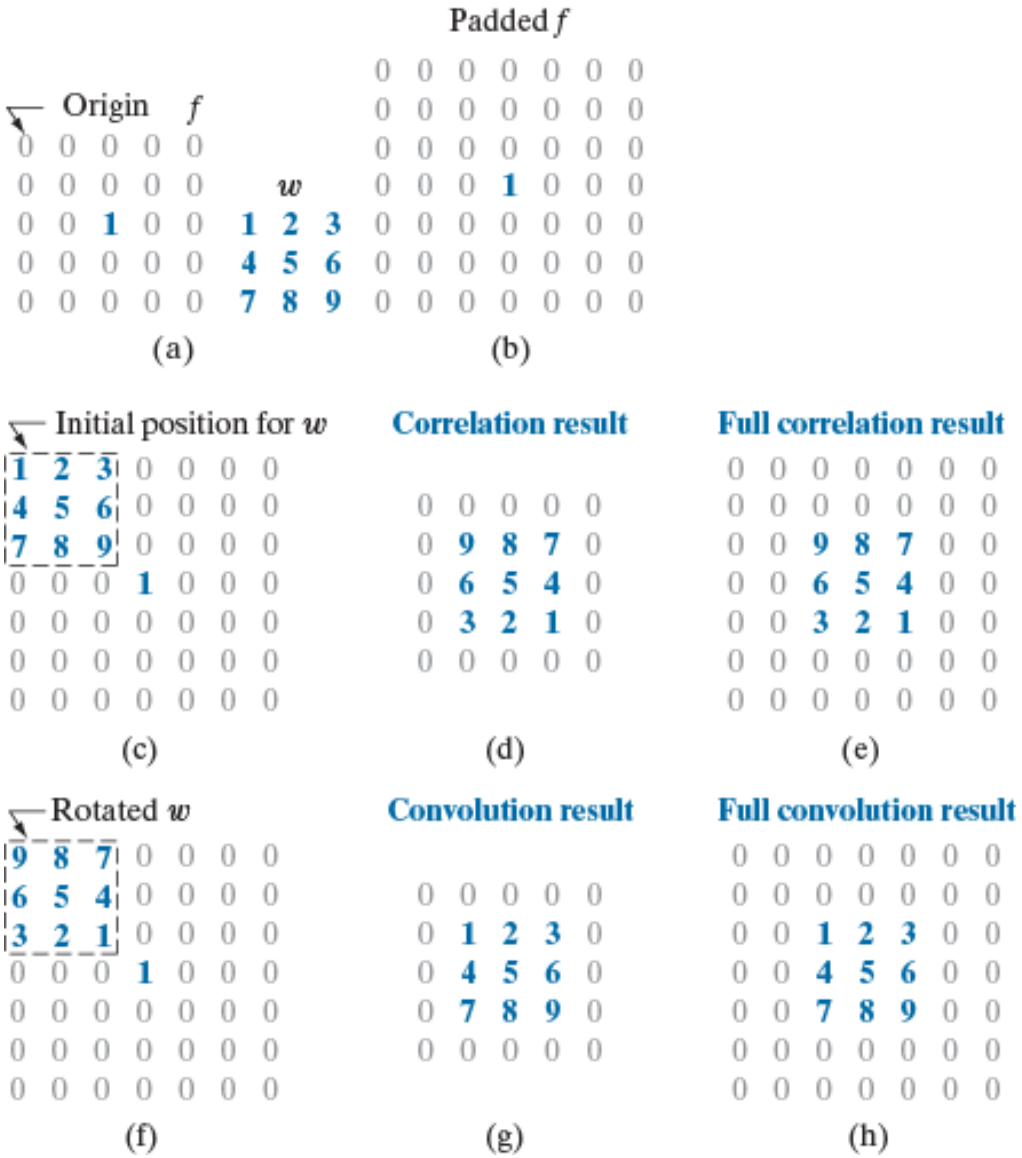
Correlation & Convolution



Correlation & Convolution



Correlation & Convolution





Convolution Operator

- Properties of convolution
  - commutativity

$$f * g = g * f$$

  - associativity

$$(f * g) * h = f * (g * h)$$

  - linearity

$$f * (\alpha g + \beta h) = \alpha(f * g) + \beta(f * h)$$

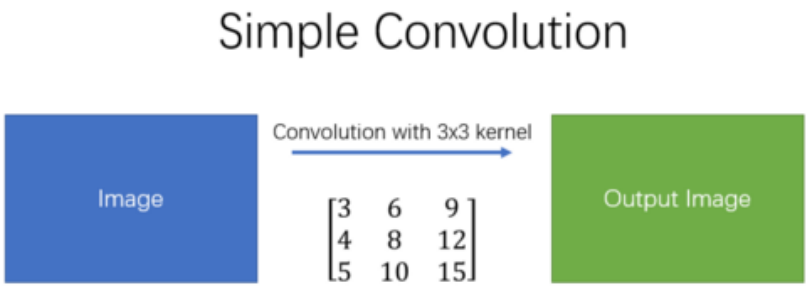
  - relationship with Fourier transform

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$$
$$\mathcal{F}(f \cdot g) = \mathcal{F}(f) * \mathcal{F}(g)$$

Separable Convolution

- Separable kernel

$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$



Each pixel 9 multiplications



Each pixel 6 multiplications

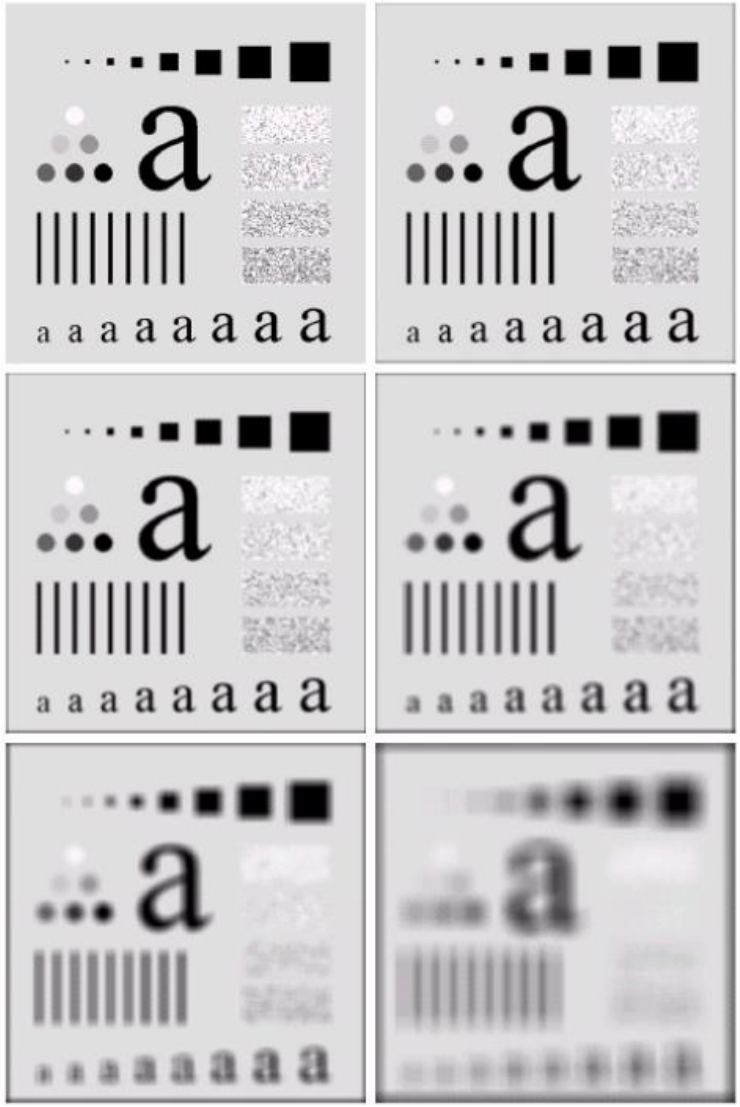
Smoothing Spatial Filters

- One of the simplest spatial filtering operations we can perform is a smoothing operation
  - Simply **average** all of the pixels in a neighborhood around a central value
  - Especially useful in removing noise from images
  - Also useful for highlighting gross detail

$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

Smoothing: The Effect of Kernel Size

- The image at the top left is an original image of size 500*500 pixels
- The subsequent images show the image after filtering with an averaging filter of increasing sizes
  - 3, 5, 9, 15 and 35
- Notice how detail begins to disappear



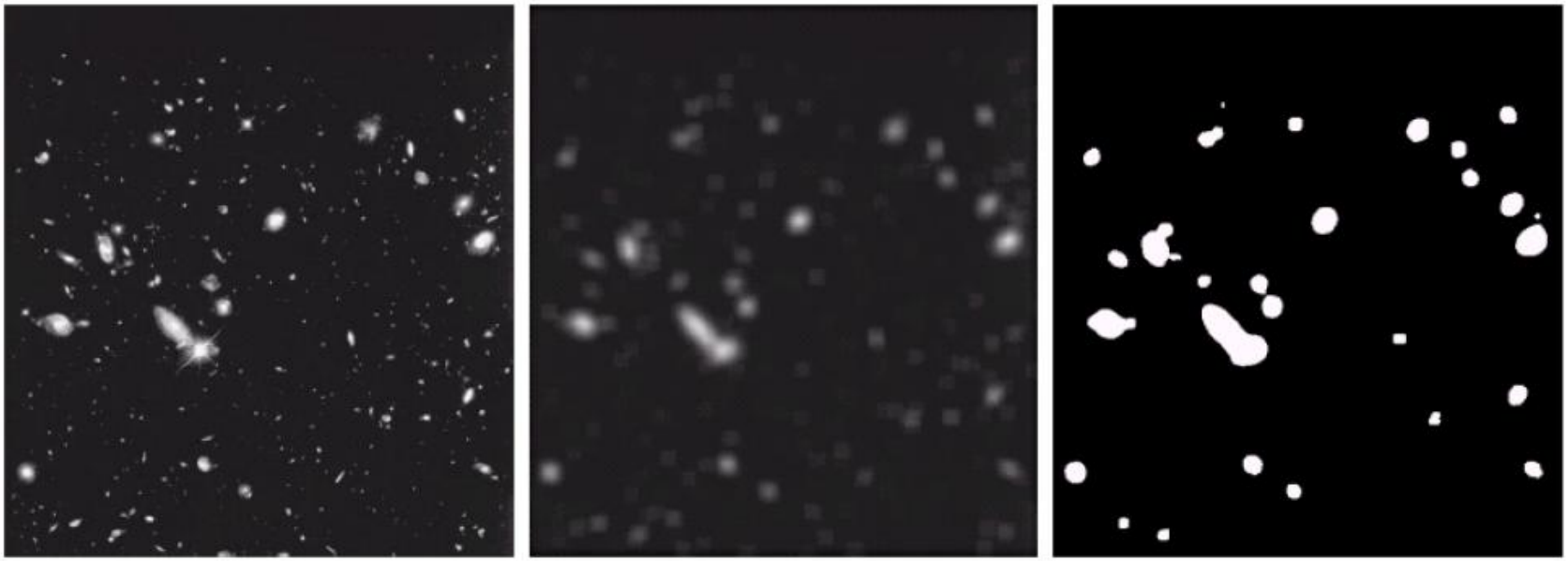
Weighted Smoothing Filters

- More effective smoothing filters can be generated by allowing different pixels in the neighborhood different weights in the averaging function
  - Pixels closer to the central pixel are more important
  - Often referred to as a weighted averaging

$1/16$	$2/16$	$1/16$
$2/16$	$4/16$	$2/16$
$1/16$	$2/16$	$1/16$

Smoothing Before Thresholding

By smoothing the original image we get rid of lots of the finer detail which leaves only the gross features for thresholding



Original Image | Smoothed Image | Thresholded Image



Effect of Low Pass Filter on White Noise

Let f be an observed instance of the image f_0 corrupted by noise w:

$$f = f_0 + w$$

with noise samples having mean value $E[w(n)]=0$ and being uncorrelated with respect to location:

$$E[w(m)w(n)] = \begin{cases} \sigma^2, & m = n \\ 0, & m \neq n \end{cases}$$

Effect of Low Pass Filter on White Noise

Applying a low pass filter h (e.g. an average filter) by convolution to the degraded image:

$$g = h * f = h * (f_0 + w) = h * f_0 + h * w$$

The expected value of the output is:

$$E[g] = E[h * f_0] + E[h * w] = h * f_0 + h * E[w]$$

$$= h * f_0 + h * 0 = h * f_0$$

The noise is removed in average.

Effect of Low Pass Filter on White Noise

What happens to the standard deviation of g?

Let $g = h * f_0 + h * w = \overline{f_0} + \overline{w}$

where the bar represents filtered versions of the signals, then

$$\sigma_g^2 = E[g^2] - \left(E[g]\right)^2 = \mathrm{E}[(\overline{f_0} + \overline{w})^2] - (\overline{f_0})^2$$

$$= \mathrm{E}[(\overline{f_0})^2 + (\overline{w})^2 + 2\overline{f_0}\,\overline{w}] - (\overline{f_0})^2$$

$$= E[(\overline{w})^2] + 2E[\overline{f_0}]E[\overline{w}] = E[(\overline{w})^2]$$

Effect of Low Pass Filter on White Noise

Considering that h is an average filter, we have at pixel n:

$$\overline{w}(n) = (h * w)(n) = \frac{1}{N} \sum_{k \in \Gamma(n)} w(k)$$

Therefore,

$$E[(\overline{w}(n))^2] = E\left[\left(\frac{1}{N} \sum_{k \in \Gamma(n)} w(k)\right)^2\right]$$

Effect of Low Pass Filter on White Noise

$$E\left[\left(\frac{1}{N}\sum_{k\in\Gamma(n)} w(k)\right)^2\right]$$

$$=\frac{1}{N^2}\sum_{k\in\Gamma(n)} E\left[\{w(k)\}^2\right]$$ → Sum of squares

$$+\frac{2}{N^2}\sum_{l\in\Gamma(n)}\sum_{\substack{m\in\Gamma(n)\\ m\neq l}} E\left[w(n-l)w(n-m)\right]$$ → Cross products

Effect of Low Pass Filter on White Noise

Sum of squares

$$\frac{1}{N^2}\sum_{k\in\Gamma(n)} E\left[\left\{w(k)\right\}^2\right] = \frac{1}{N^2}\sum_{k\in\Gamma(n)} \sigma^2$$

Cross products (uncorrelated as $m \neq l$)

$$+\frac{2}{N^2}\sum_{l\in\Gamma(n)}\sum_{\substack{m\in\Gamma(n)\\ m\neq l}} E\left[w(n-l)w(n-m)\right] = 0$$

Effect of Low Pass Filter on White Noise

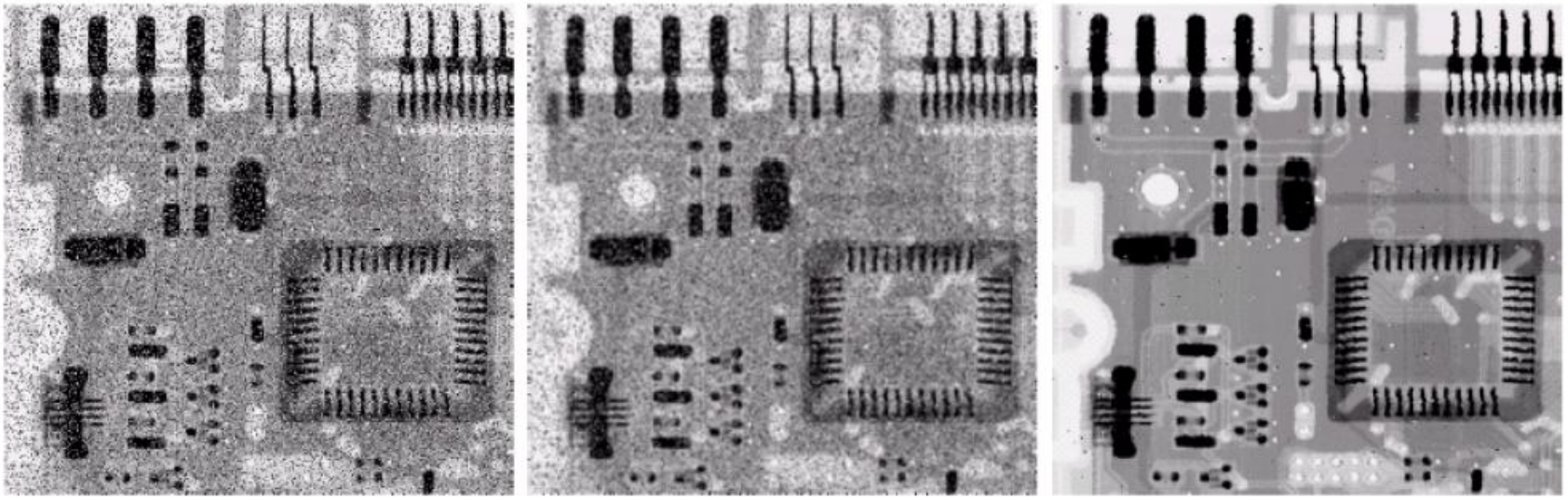
Finally, substituting the partial results:

$$\sigma_g^2 = E\left[\left(\frac{1}{N}\sum_{k\in\Gamma(n)} w(k)\right)^2\right] = \frac{1}{N^2}\sum_{k\in\Gamma(n)} \sigma^2$$

$$= \frac{1}{N^2} N\sigma^2 = \frac{\sigma^2}{N}$$

The effect of the noise is reduced.
This processing is not optimal as it also smoothes image edges.

Averaging Filter vs. Median Filter



Original Image With Noise

Image After Averaging Filter

Image After Median Filter

- Filtering is often used to remove noise from images
- Sometimes a median filter works better than an averaging filter

Spatial Smoothing

- Spatial smoothing may be viewed as a process for estimating the value of a pixel from its neighbors.
- What is the value that “best” approximates the intensity of a given pixel given the intensities of its neighbors?
- We have to define “best” by establishing a criterion.

Spatial Smoothing

- A standard criterion is the sum of squares differences.

$$E = \sum_{i=1}^{N} \left[x(i) - m \right]^2 \quad \Leftrightarrow \quad m = \arg\min_{m} \left\{ \sum_{i=1}^{N} \left[x(i) - m \right]^2 \right\}$$

$$\frac{\partial E}{\partial m} = 0 \quad \Leftrightarrow \quad -2 \sum_{i=1}^{N} \left(x(i) - m \right) = 0 \quad \Leftrightarrow \quad \sum_{i=1}^{N} x(i) = \sum_{i=1}^{N} m$$

$$\Leftrightarrow \quad \sum_{i=1}^{N} x(i) = Nm \quad \Leftrightarrow \quad m = \frac{1}{N} \sum_{i=1}^{N} x(i)$$

The average value

Spatial Smoothing

- Another criterion is the sum of absolute differences.

$$E = \sum_{i=1}^{N} |x(i) - m| \quad \Leftrightarrow m = \arg\min_{m} \left\{ \sum_{i=1}^{N} |x(i) - m| \right\}$$

$$\frac{\partial E}{\partial m} = 0 \Leftrightarrow -\sum_{i=1}^{N} sign\big(x(i) - m\big) = 0, \quad sign(x) = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \\ -1 & x < 0 \end{cases}$$

There must be equal in quantity positive and negative values.

$$m = \text{median}\{x(i)\}$$

If 51% of samples are correct and 49% outliers, the median still finds the right level!

Median Filter

- The median filter is non linear:

  median{x+y}≠median{x}+median{y}

- It works well for impulse noise(e.g. salt and pepper).
- It requires sorting of the image values.
- It preserves the edges better than an average filter in the case of impulse noise
- It is robust to impulse noise at 50%

Median Filtering: An Edge Example

Example

x[n]	1	1	1	1	1	2	2	2	2	2

edge

Impulse noise

x[n]	1	3	1	1	1	2	3	2	2	3

Median (N=3)

x[n]	-	1	1	1	1	2	2	2	2	-

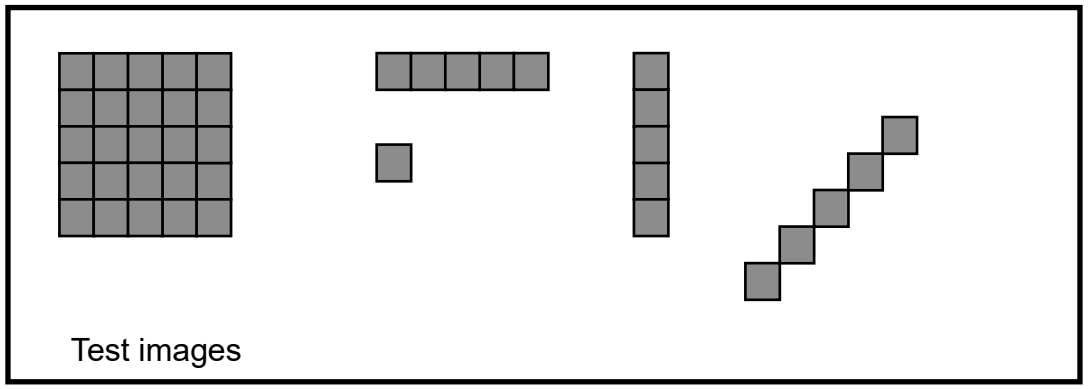
Average (N=3)

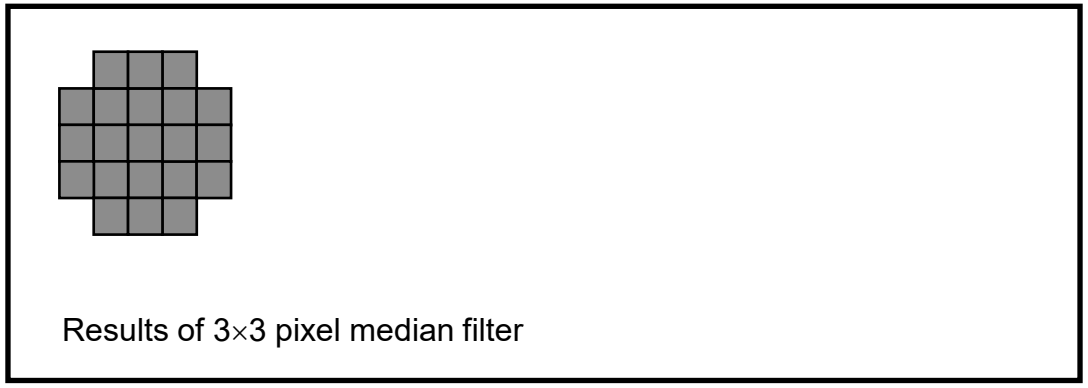
x[n]	-	1.7	1.7	1	1.3	2	2.3	2.3	2.3	-

The edge is smoothed

Median Filtering: Structure Loss

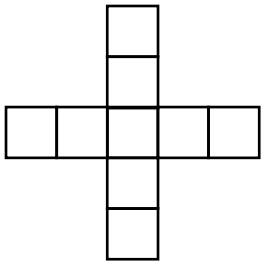
- Caution: points, thin lines and corners are erased by the median filter

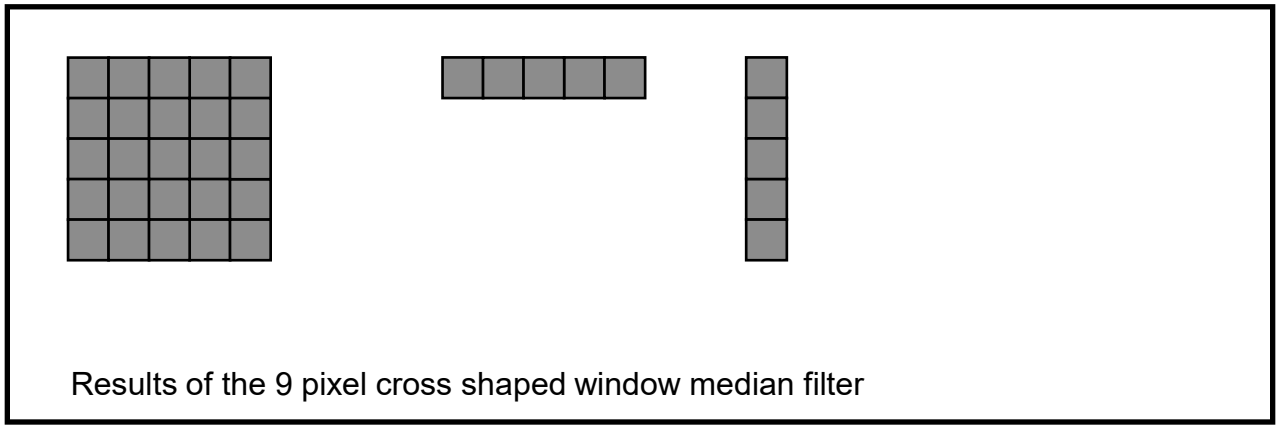




The Effect of Window Shape

- Cross shaped window can correct some of the problems





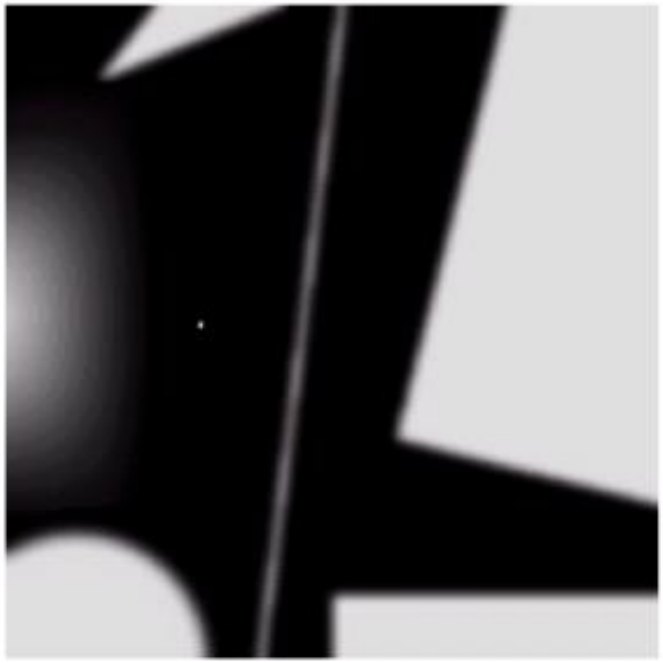
Results of the 9 pixel cross shaped window median filter

Sharpening Spatial Filters

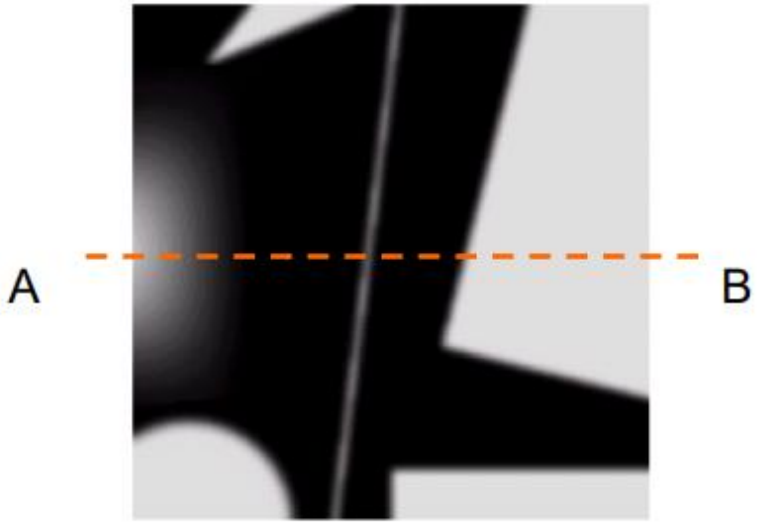
- Previously we have looked at smoothing filters which remove fine detail
- Sharpening spatial filters seek to highlight fine detail
  - Remove blurring from images
  - Highlight edges
- Sharpening filters are based on spatial differentiation

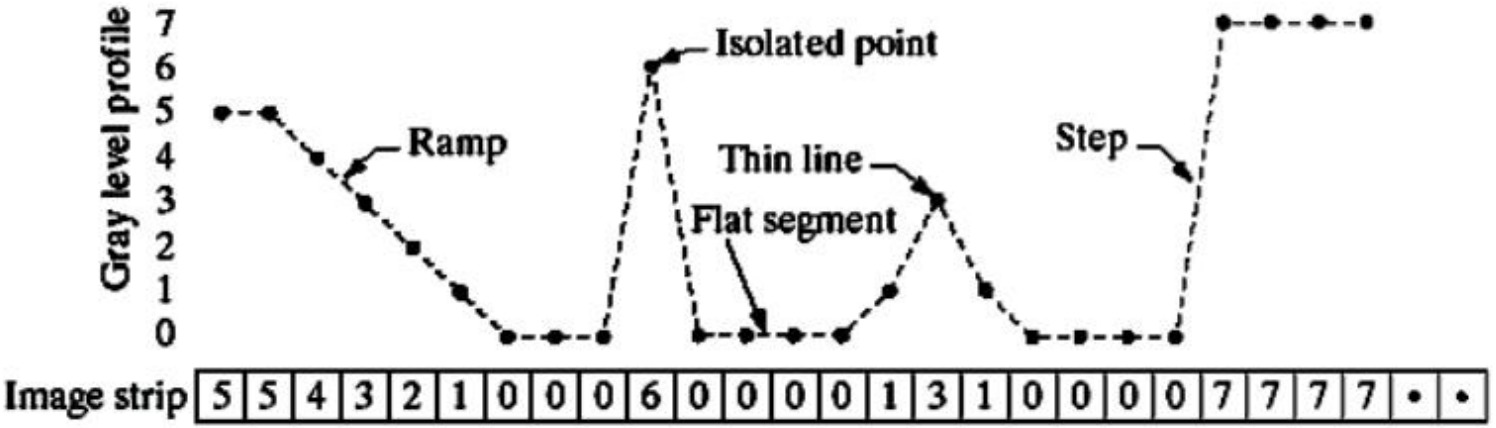
Spatial Differentiation

- Differentiation measures the rate of change of a function
- Let's consider a simple 1-dimensional example



Spatial Differentiation: A 1D Profile





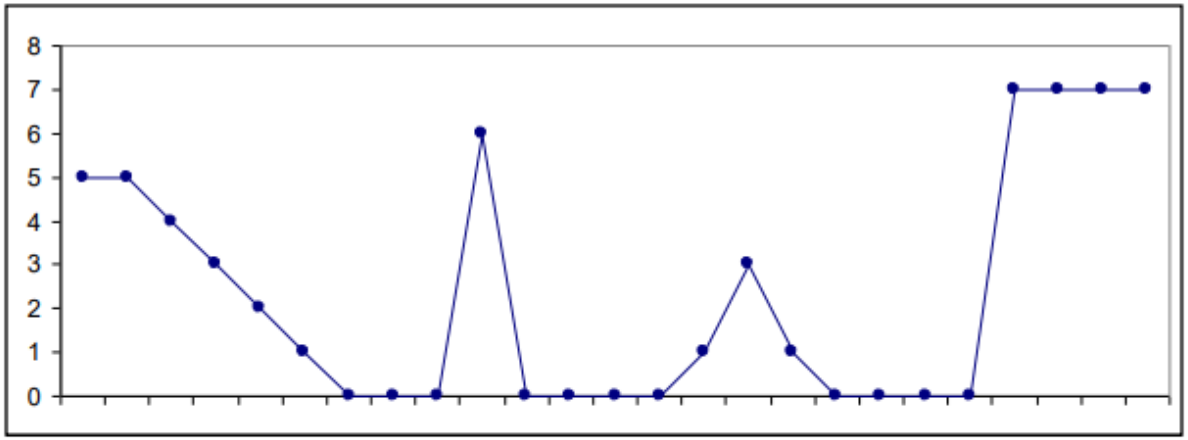
1st Derivative

- Discrete approximation of the 1st derivative

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

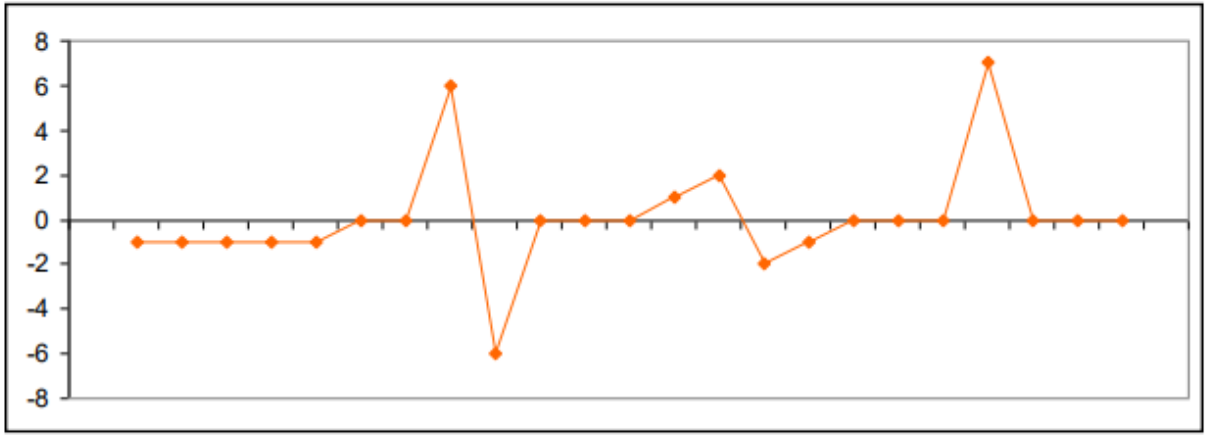
- It is just the difference between subsequent values and measures the rate of change of the function

First Difference: A Detailed Example



5	5	4	3	2	1	0	0	0	6	0	0	0	0	1	3	1	0	0	0	7	7	7	7

	0	-1	-1	-1	-1	-1	0	0	6	-6	0	0	0	1	2	-2	-1	0	0	0	7	0	0	0



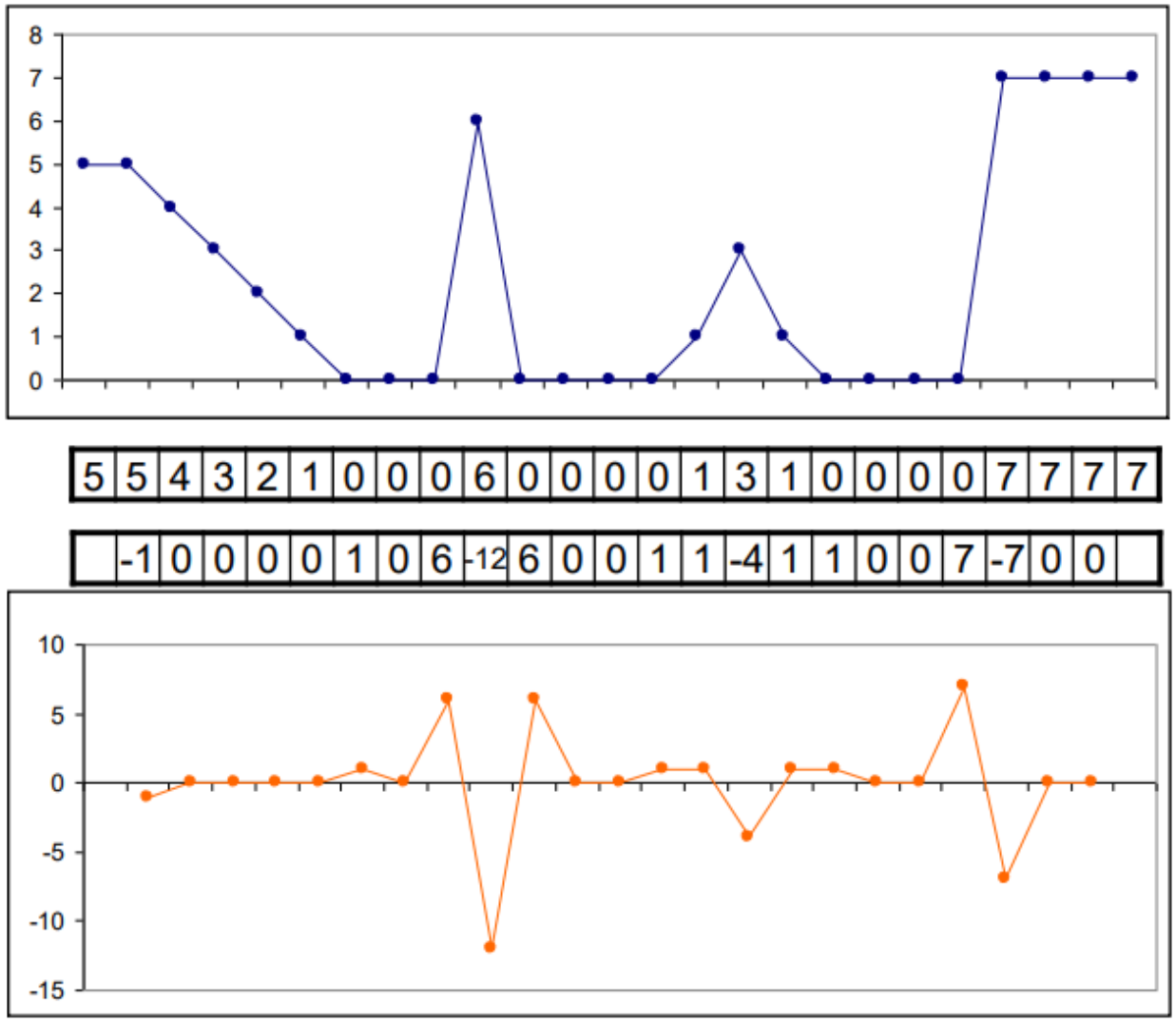


2nd Derivative

- Discrete approximation of the 2nd derivative:

$$\frac{\partial^2 f}{\partial^2 x} = f(x+1) + f(x-1) - 2f(x)$$

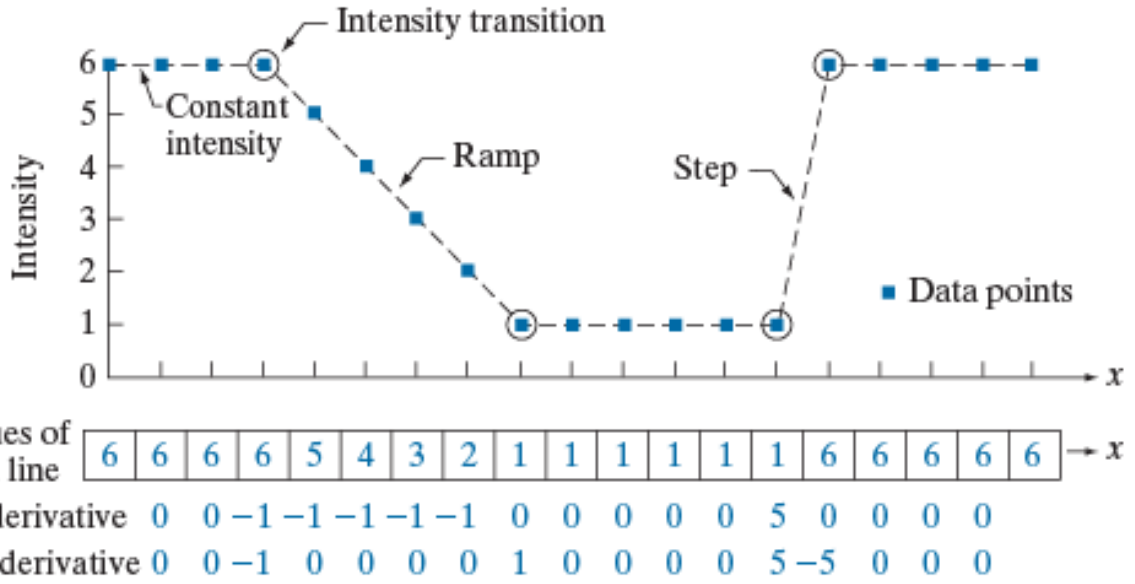
Second Difference: A Detailed Example



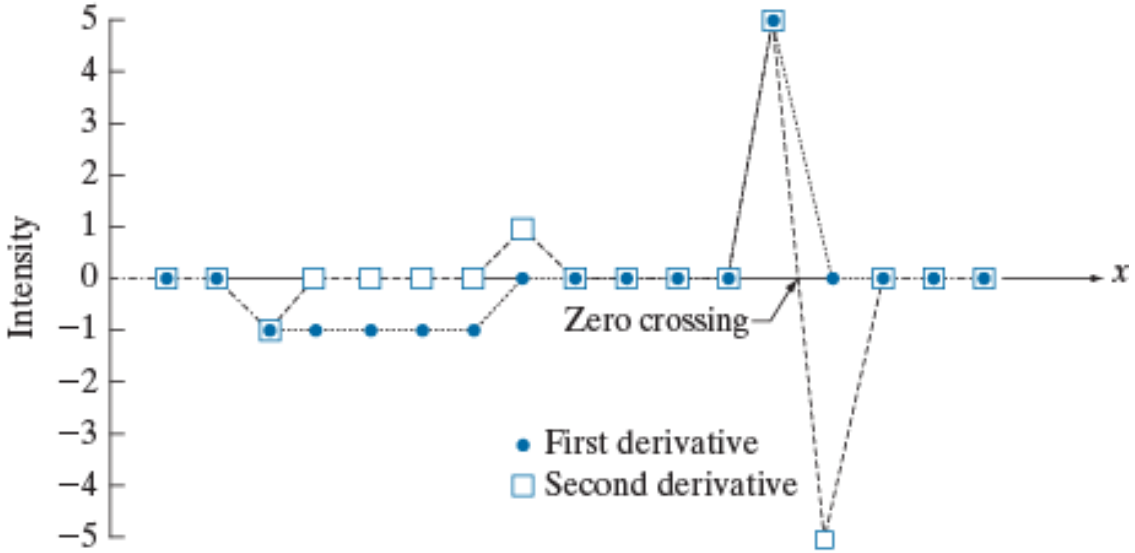
Using 2nd Derivatives for Image Enhancement

- Edges in images are often ramp-like transitions
  - 1st derivative is constant and produces thick edges
  - 2nd derivative zero crosses the edge (double response at the onset and end with opposite signs)

Derivatives



Values of scan line	6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6
1st derivative	0	0	−1	−1	−1	−1	−1	0	0	0	0	0	5	0	0	0	0		
2nd derivative	0	0	−1	0	0	0	0	1	0	0	0	0	5	−5	0	0	0		



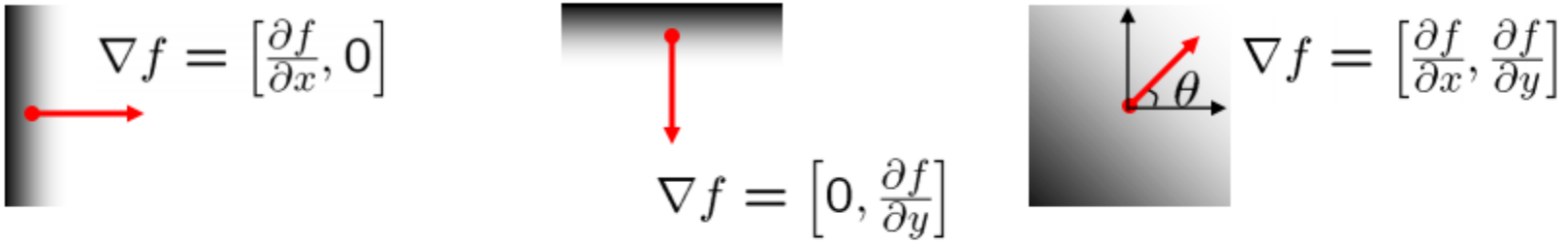
Derivative Filters Requirements

- First derivative filter output
  - Zero at constant intensities
  - Non zero at the onset of a step or ramp
  - Non zero along ramps
- Second derivative filter output
  - Zero at constant intensities
  - Non zero at the onset and end of a step or ramp
  - Zero along ramps of constant slope

1st & 2nd Derivatives

- Comparing the 1st and 2nd derivatives we can conclude the following:
  - 1st order derivatives generally produce thicker edges (if thresholded at ramp edges)
  - 2nd order derivatives have a stronger response to fine detail e.g. thin lines
  - 1st order derivatives have stronger response to gray level step
  - 2nd order derivatives produce a double response at step changes in grey level (which helps in detecting zero crossings)

The Image Gradient

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$
- 

The gradient points in the direction of most rapid increase in intensity.

Gradient direction $\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$

The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Gradient Components and Magnitude





$\|\nabla f\|$

$\frac{\partial f}{\partial x}$





$\frac{\partial f}{\partial y}$



Using 1st Derivatives For Image Enhancement

$$\nabla f = \begin{bmatrix} G_x & G_y \end{bmatrix}^T = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \end{bmatrix}^T$$

- Although the derivatives are linear operators, the gradient magnitude is not.
- Also, the partial derivatives are not rotation invariant (isotropic).
- The magnitude of the gradient vector is isotropic.

Sobel Operators

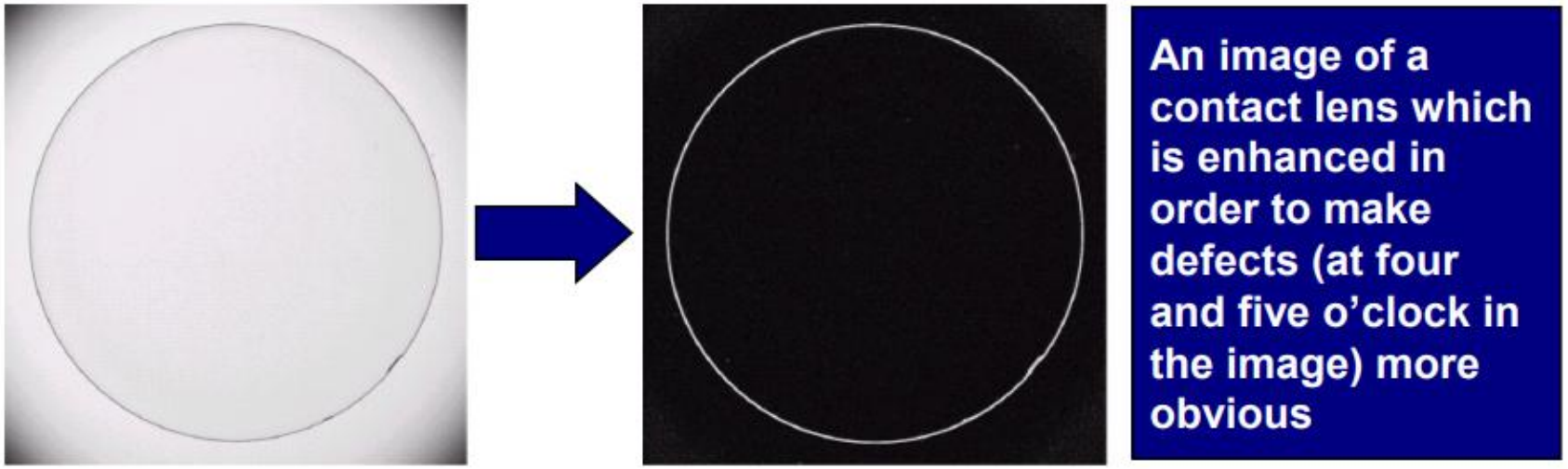
- Sobel operators introduce the idea of smoothing by giving more importance to the center point:

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

- Note that the coefficients sum to 0 to give a 0 response at areas of constant intensity.

Sobel: An Inspection Example



- Sobel gradient aids to eliminate constant or slowly varying shades of gray and assist automatic inspection.
- It also enhances small discontinuities in a flat gray field.



Using 2nd Derivatives for Image Enhancement

- A common sharpening filter is the Laplacian
  - Isotropic
    - Rotation invariant: Rotating the image and applying the filter is the same as applying the filter and then rotating the image.
    - In other words, the Laplacian of a rotated image is the rotated Laplacian of the original image.
  - One of the simplest sharpening filters
  - We will look at a digital implementation

$$\nabla^2 f = \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y}$$

The Laplacian

$$\nabla^2 f = \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y}$$

$$\frac{\partial^2 f}{\partial^2 x} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial^2 y} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

- Laplacian filter is a linear operator

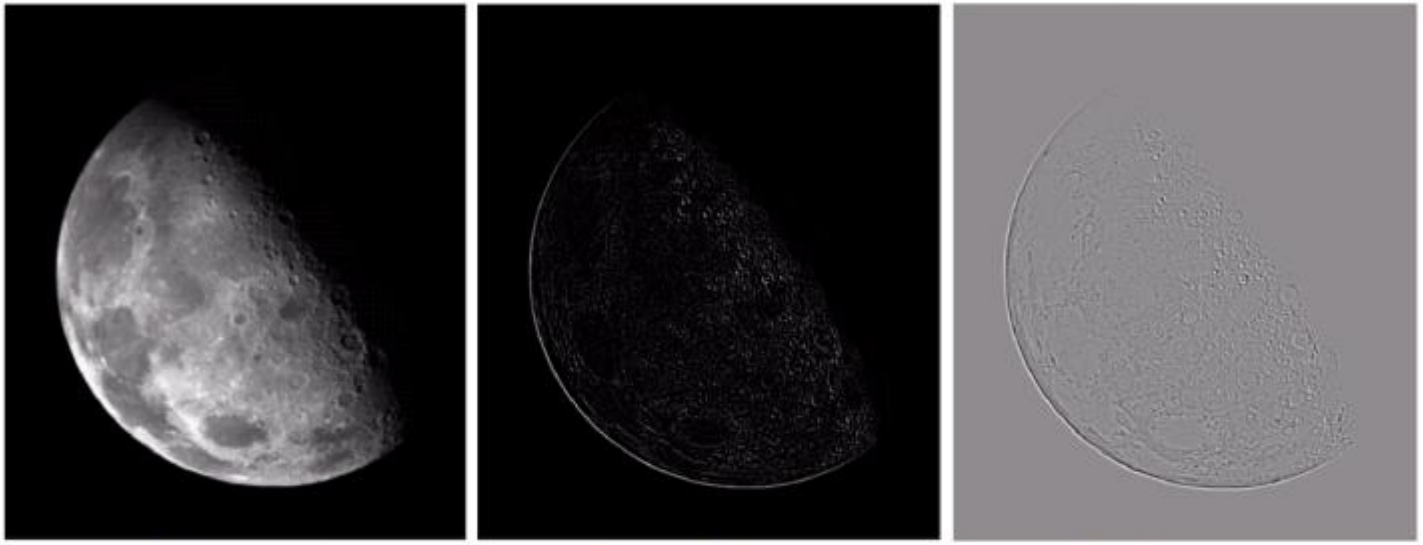
The Laplacian

$$\nabla^2 f = -4f(x,y)$$
$$+f(x+1,y)+f(x-1,y)$$
$$+f(x,y+1)+f(x,y-1)$$

0	1	0
1	-4	1
0	1	0

Laplacian Response and Display

- Applying the Laplacian to an image we get a new image that highlights edges and other discontinuities



Original Image

Laplacian Filtered Image

Laplacian Filtered Image Scaled for Display

The Laplacian

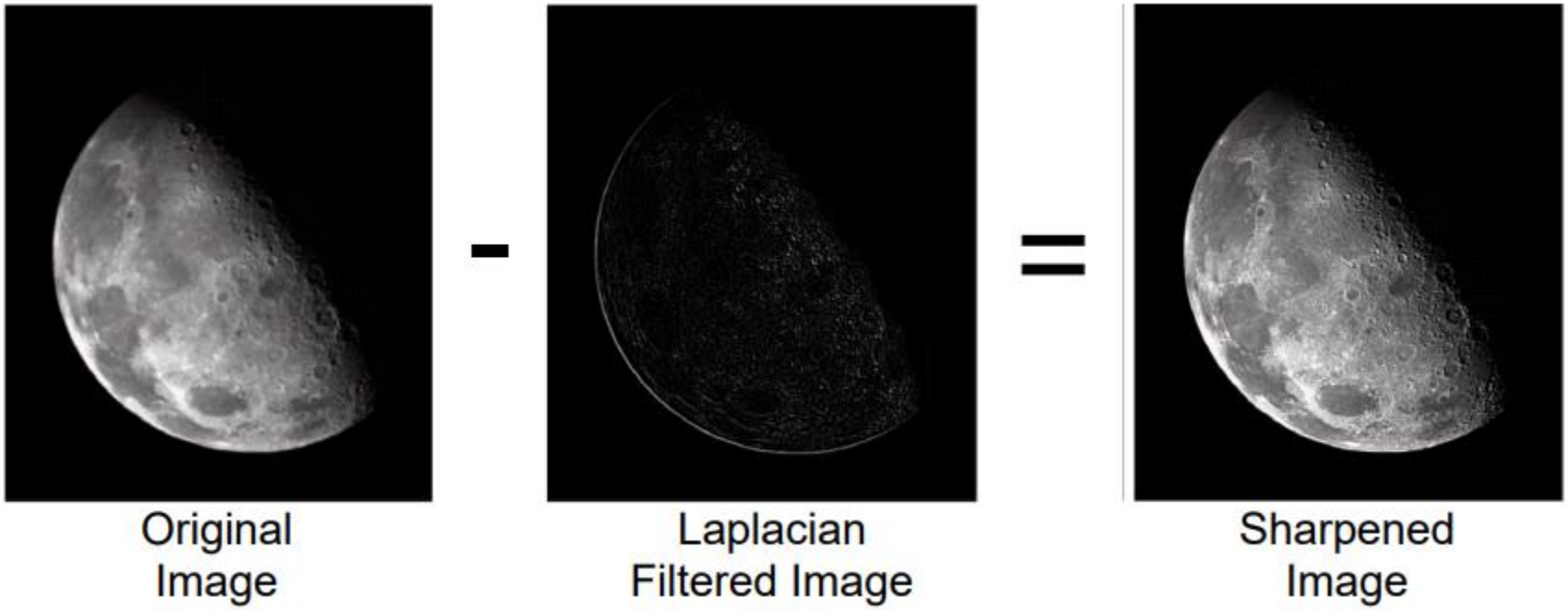
- The result of a Laplacian filtering is not an enhanced image
- We have to do more work
- Subtract the Laplacian result from the original image to generate our final sharpened enhanced image



Laplacian
Filtered Image
Scaled for Display

$$g(x,y) = f(x,y) - \nabla^2 f$$

Laplacian Image Enhancement



In the final, sharpened image, edges and fine detail are much more obvious

Laplacian Enhancement: A Comparison



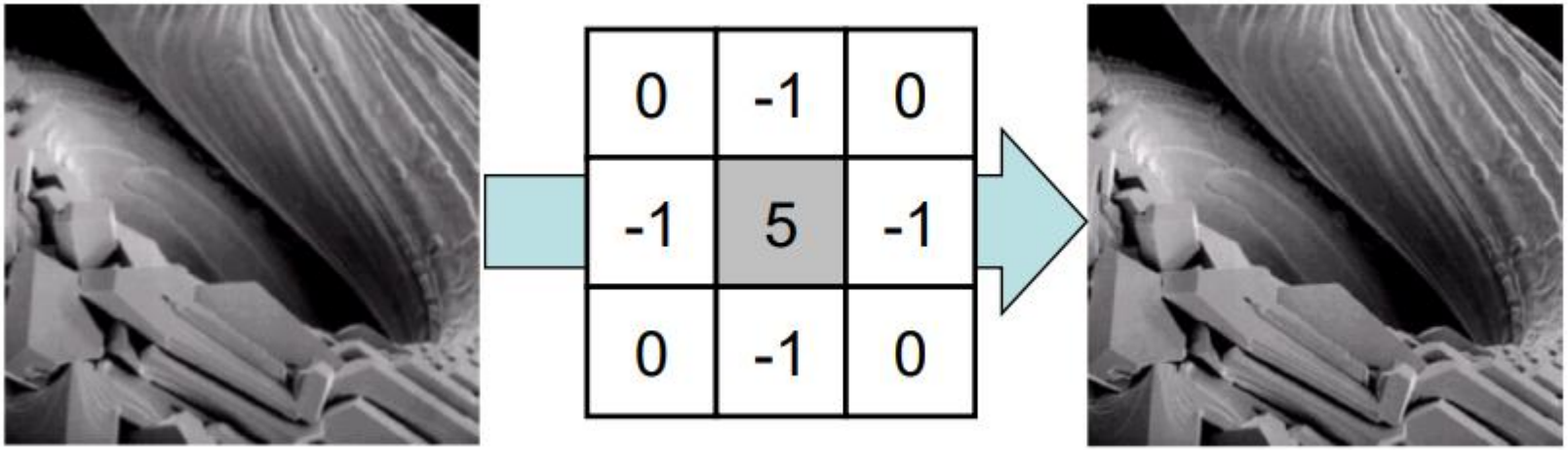
Simplified Image Enhancement

- The entire enhancement can be combined into a single filtering operation:

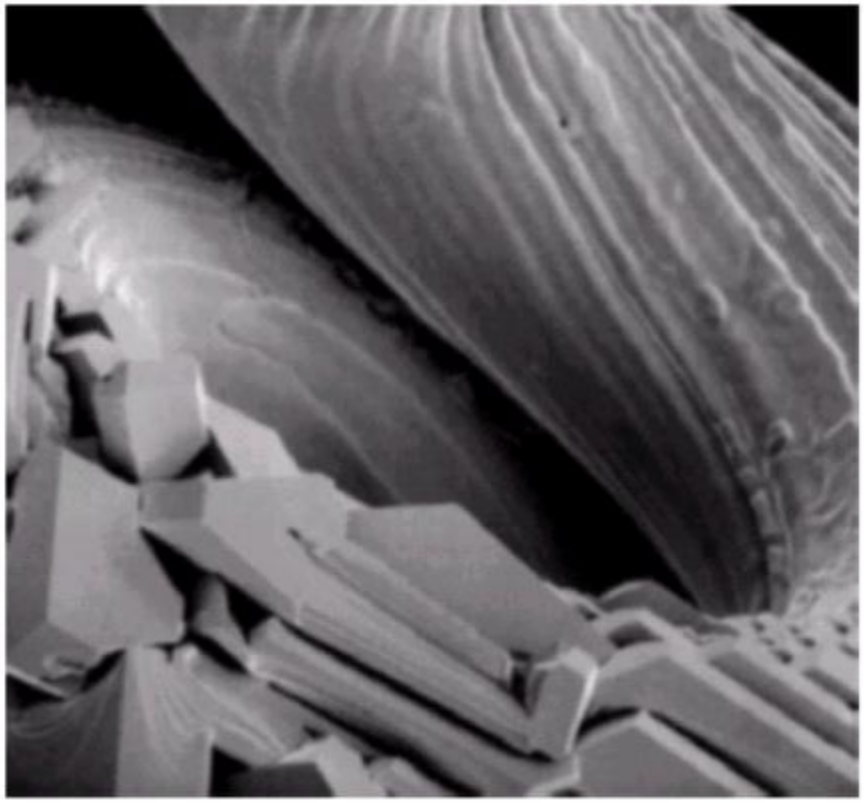
$$
\begin{aligned}
g(x,y) &= f(x,y) - \nabla^2 f \\
&= 5f(x,y) - f(x+1,y) - f(x-1,y) \\
&\quad - f(x,y+1) - f(x,y-1)
\end{aligned}
$$

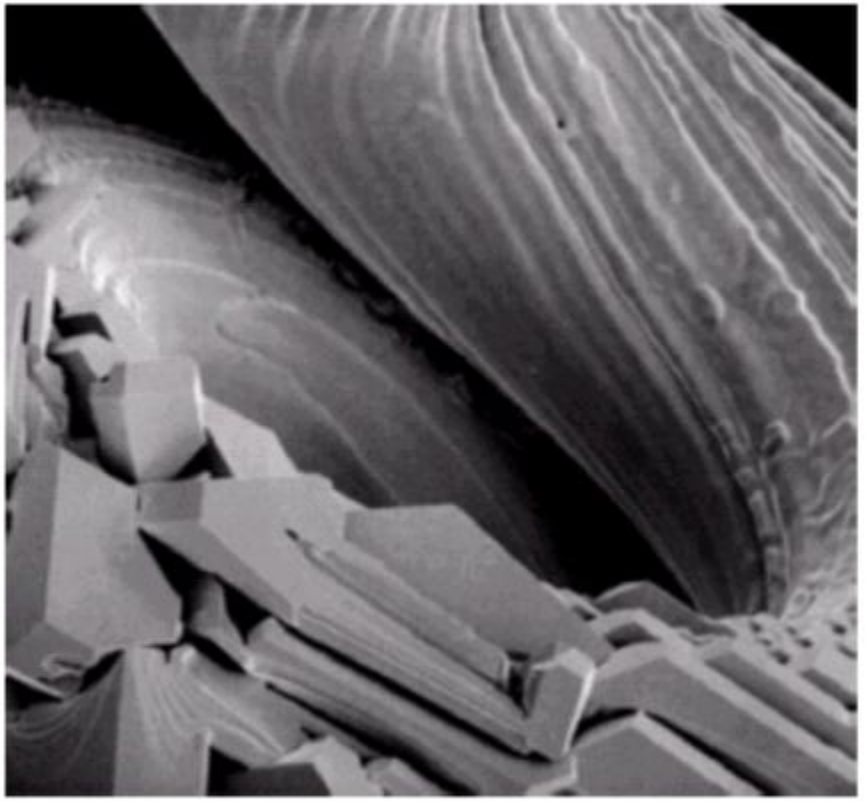
A Single Sharpening Kernel

- This gives us a new filter which does the whole job in one step



Sharpening: Fine Detail





Laplacian Kernel Variants

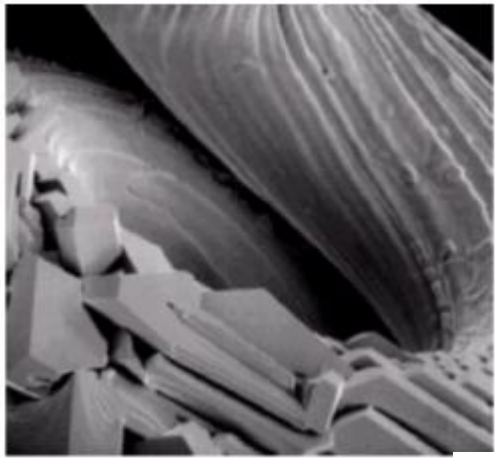
- There are lots of slightly different versions of the Laplacian that can be used:

0	1	0
1	-4	1
0	1	0

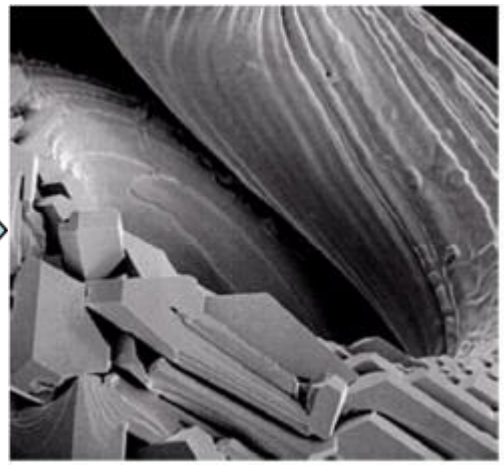
Standard Laplacian

1	1	1
1	-8	1
1	1	1

Variant of Laplacian



-1	-1	-1
-1	9	-1
-1	-1	-1



Unsharp masking（钝化掩蔽）

- Subtracts an unsharped (smooth) image from the original image f (x,y).
  - Blur the image

$$b(x,y)=Blur\{f(x,y)\}$$

  - Subtract the blurred image from the original (the result is called the mask)

$$g_{mask}(x,y)=f(x,y)-b(x,y)$$

  - Add the mask to the original

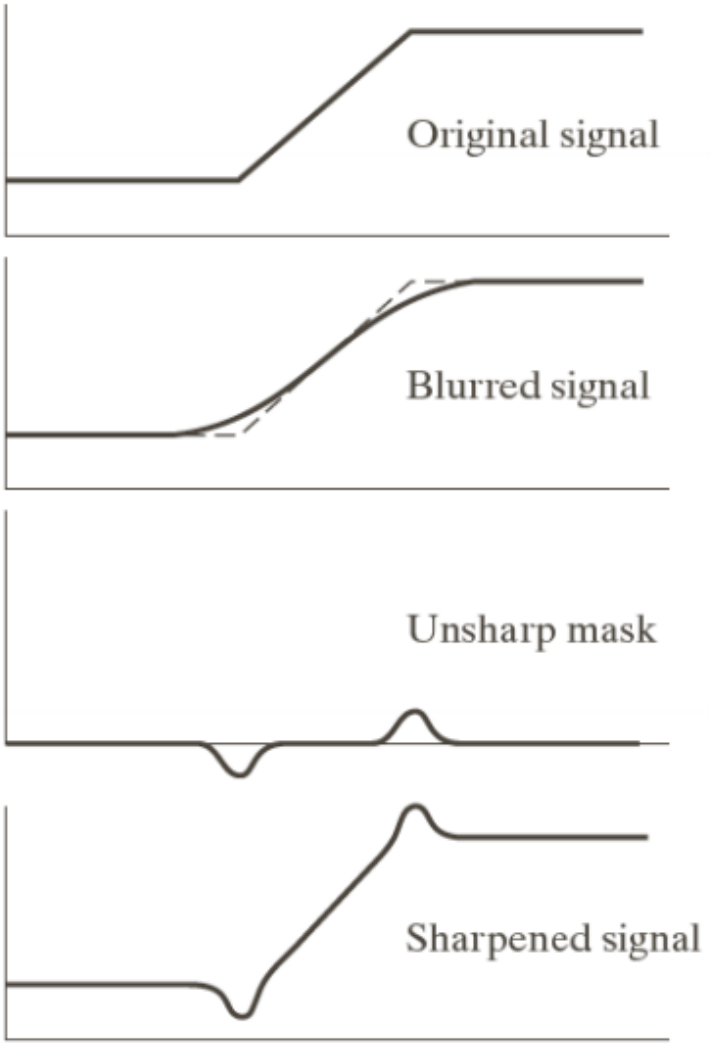
$$g(x,y)=f(x,y)+k\,g_{mask}(x,y),\ k \text{ being non negative}$$

Unsharp Masking: A Signal Profile

Sharpening mechanism

If $k>1$, the process is referred to as **highboost filtering**

高提升滤波



Unsharp Masking: Gain Comparison

Original image



Blurred image
(Gaussian 5x5, σ=3)



Mask



Unsharp masking (k=1)

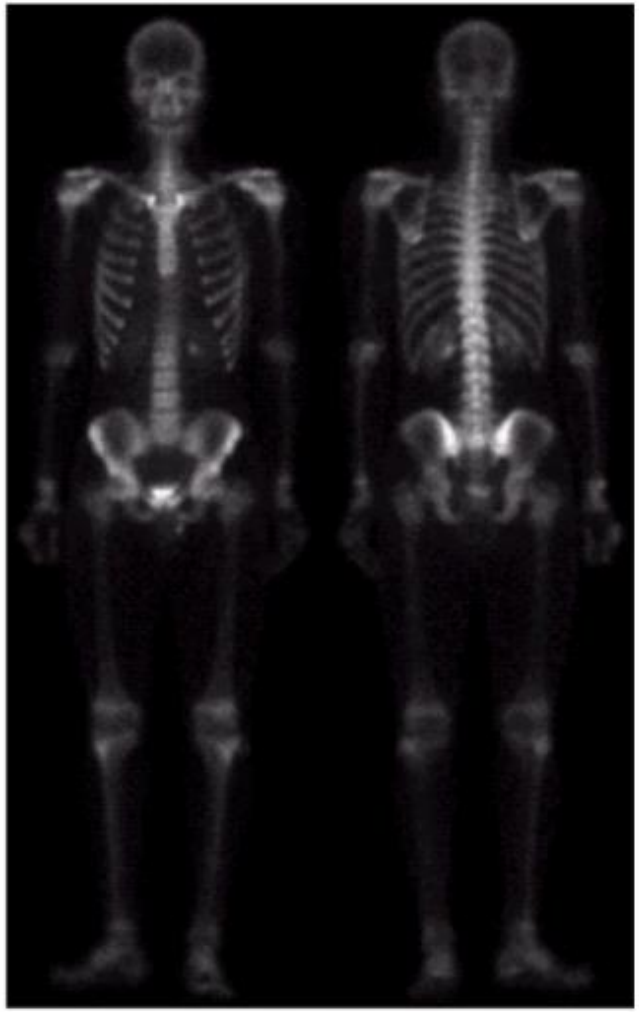


Highboost filtering (k=4.5)

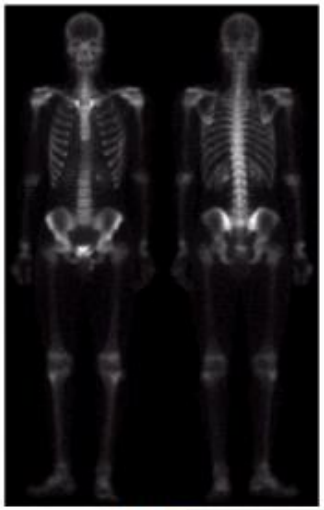


Combining Enhancement Methods

- Successful image enhancement is typically not achieved using a single operation
- Rather we combine a range of techniques in order to achieve a final result
- This example will focus on enhancing the bone scan to the right



Combined Enhancement: Detail and Edges



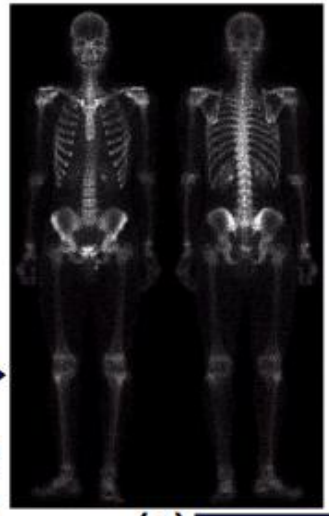
(a)

Laplacian filter of bone scan (a)



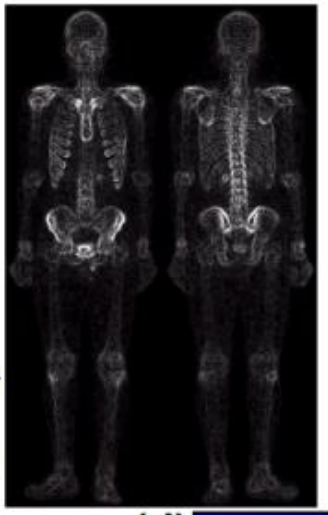
(b)

Sharpened version of bone scan achieved by subtracting (a) and (b)



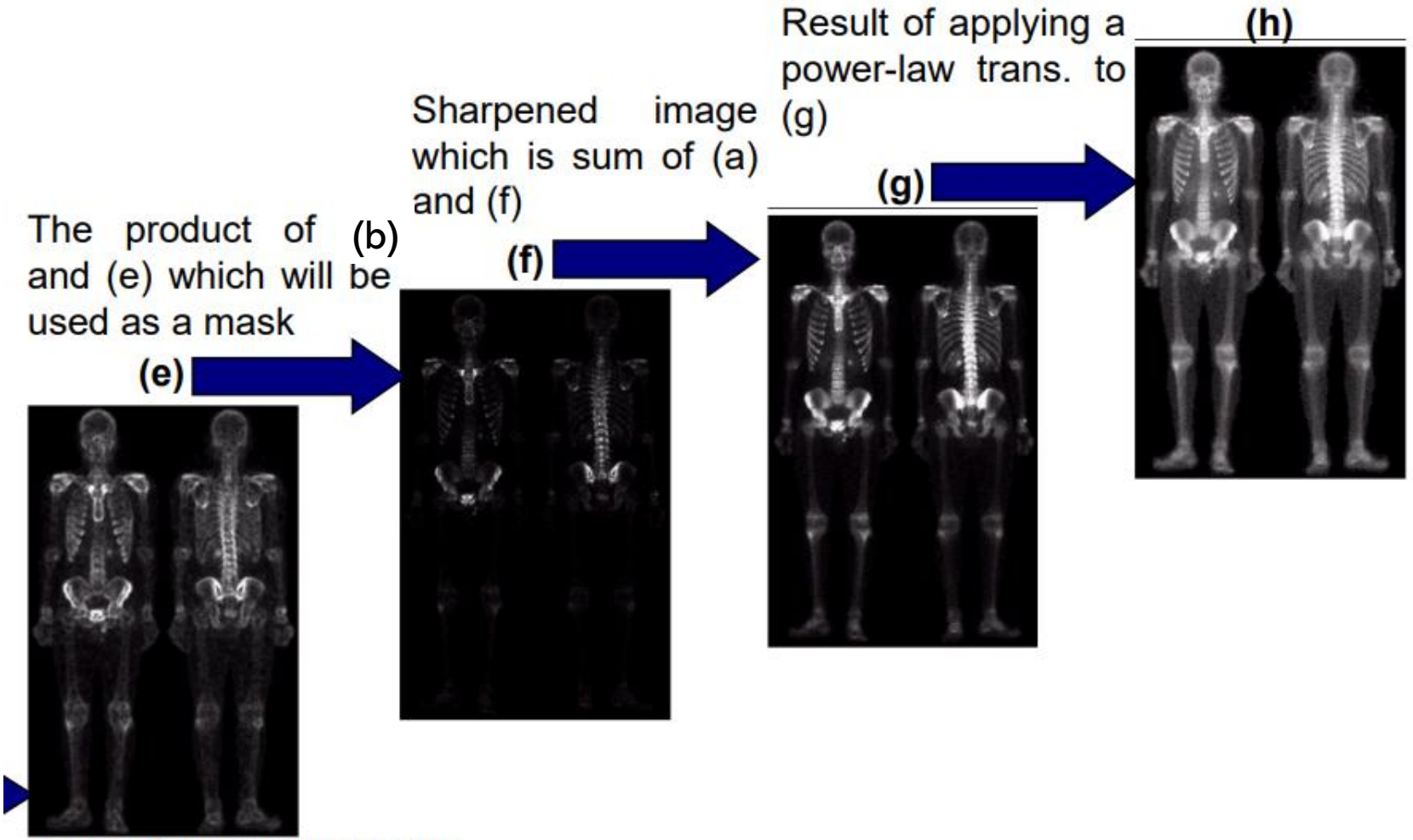
(c)

Sobel filter of bone scan (a)



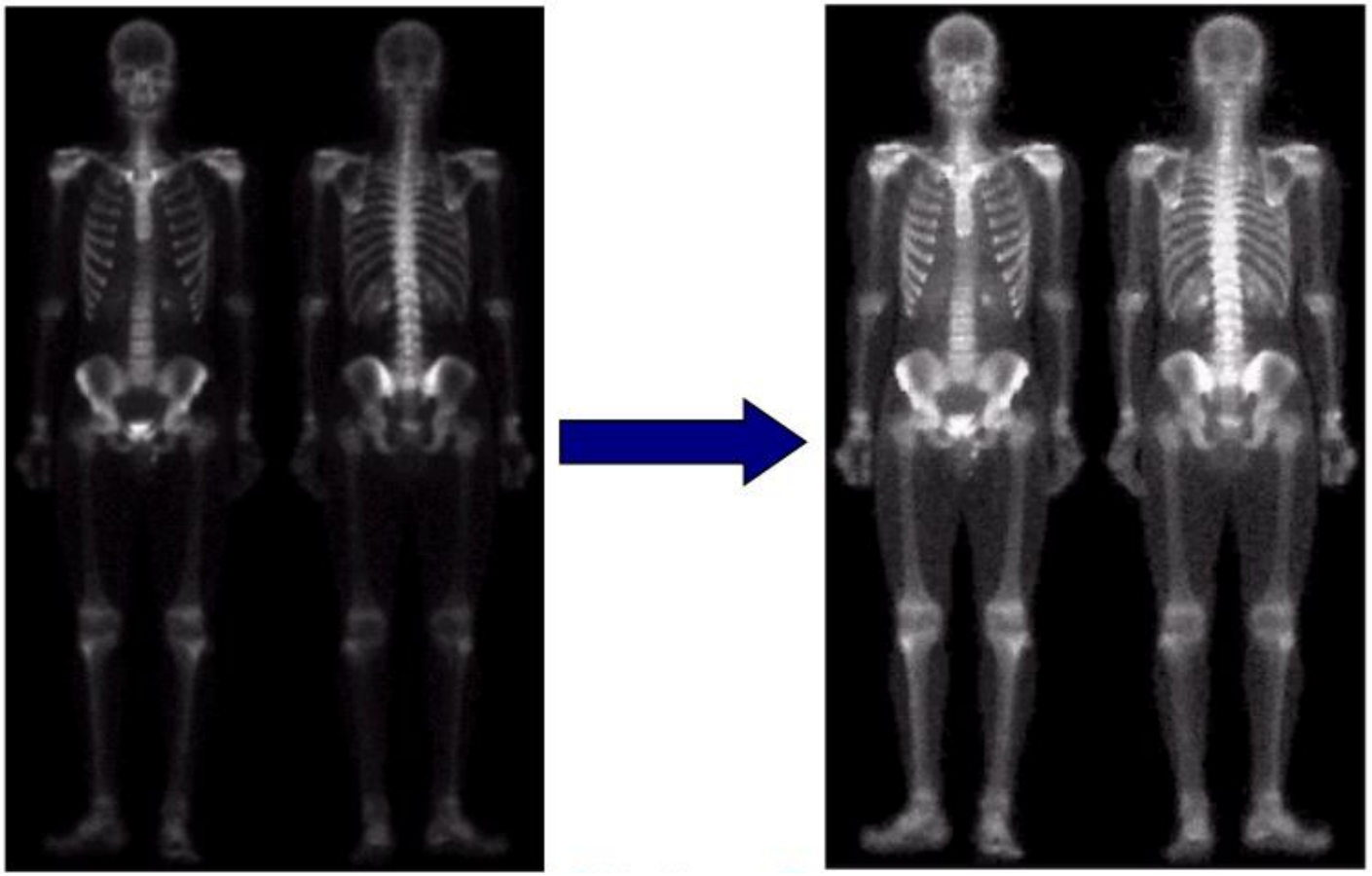
(d)

Combined Enhancement: Fusion



Combined Enhancement: Final Comparison

Compare the original and final images

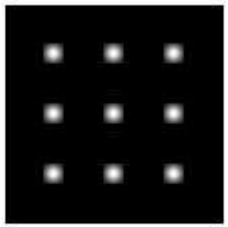


Homework

- 生成一个黑色图像S，其中仅有9个pixel是白色的。生成一个PSF，并用它来卷积S，观察点扩展函数的效果

```
f=zeros(100,100);
for i=20:30:80;
    for j=20:30:80
        f(i,j)=1;
    end;
end;
figure, imshow(f);   %生成有9个白色像素的100*100图像
psf=fspecial('gaussian',[10,10],3); %用高斯模糊生成PSF
f2=conv2(f,psf);          %卷积
figure,imshow(f2,[ ]);
```





Homework

- 2.阅读imnoise2，并运用它产生不同概率密度分布的随机数，观察其概率密度分布。

```
>> help imnoise2
 IMNOISE2 Generates an array of random numbers with specified PDF.
    R = IMNOISE2(TYPE, M, N, A, B) generates an array, R, of size
    M-by-N, whose elements are random numbers of the specified TYPE
    with parameters A and B. If only TYPE is included in the
    input argument list, a  single random number of the specified
    TYPE and default parameters shown below is generated. If only
    TYPE, M, and N are provided, the default parameters shown below
    are used.  If M = N = 1, IMNOISE2 generates a single random
    number of the specified TYPE and parameters A and B.

    Valid values for TYPE and parameters A and B are:

    'uniform'      Uniform random numbers in the interval (A, B).
                   The default values are (0, 1).
    'gaussian'     Gaussian random numbers with mean A and standard
                   deviation B. The default values are A = 0, B = 1.
```

Homework

- 2.阅读imnoise2，并运用它产生不同概率密度分布的随机数，观察其概率密度分布。

'salt & pepper' Salt and pepper numbers of amplitude 0 with probability Pa = A, and amplitude 1 with probability Pb = B. The default values are Pa = Pb = A = B = 0.05. Note that the noise has values 0 (with probability Pa = A) and 1 (with probability Pb = B), so scaling is necessary if values other than 0 and 1 are required. The noise matrix R is assigned three values. If R(x, y) = 0, the noise at (x, y) is pepper (black). If R(x, y) = 1, the noise at (x, y) is salt (white). If R(x, y) = 0.5, there is no noise assigned to coordinates (x, y).

'lognormal' Lognormal numbers with offset A and shape parameter B. The defaults are A = 1 and B = 0.25.

'rayleigh' Rayleigh noise with parameters A and B. The default values are A = 0 and B = 1.

Homework

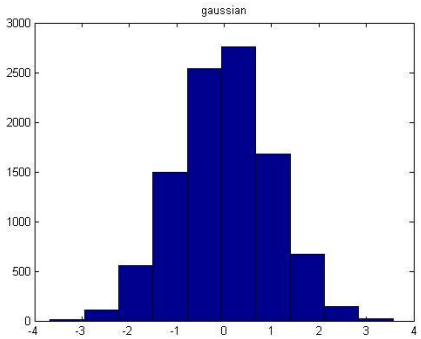
- 2.阅读imnoise2，并运用它产生不同概率密度分布的随机数，观察其概率密度分布。

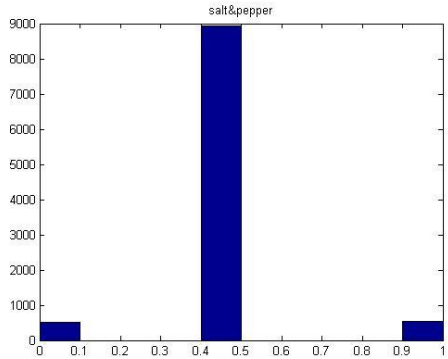
'exponential' Exponential random numbers with parameter A. The default is A = 1.

'erlang' Erlang (gamma) random numbers with parameters A and B. B must be a positive integer. The defaults are A = 2 and B = 5. Erlang random numbers are approximated as the sum of B exponential random numbers.

演示这些噪声特性

```
r1=imnoise2('uniform',10000,1,0,1)
%平均噪声
hist(r1);title('uniform');
r2=imnoise2('gaussian',10000,1,0,1
; % 高斯分布噪声，以0为原点，1为标准差
figure,hist(r2);title('gaussian');
r3=imnoise2('salt &
pepper',10000,1,0.05,0.05);%·椒盐噪声，
黑白概率各为0.05
```





Homework

- 任选一幅图像，给它分别加上imnoise所产生的噪声类型：pepper&salt和Gaussian。

```
f=imread('cameraman.tif');
imshow(f,[])
f1=imnoise(f,'gaussian',0.1); %高斯噪声
figure,imshow(f1);title('gaussian');
f2=imnoise(f,'salt & pepper',0.1); %椒盐噪声
figure,imshow(f2);title('salt & pepper');
```





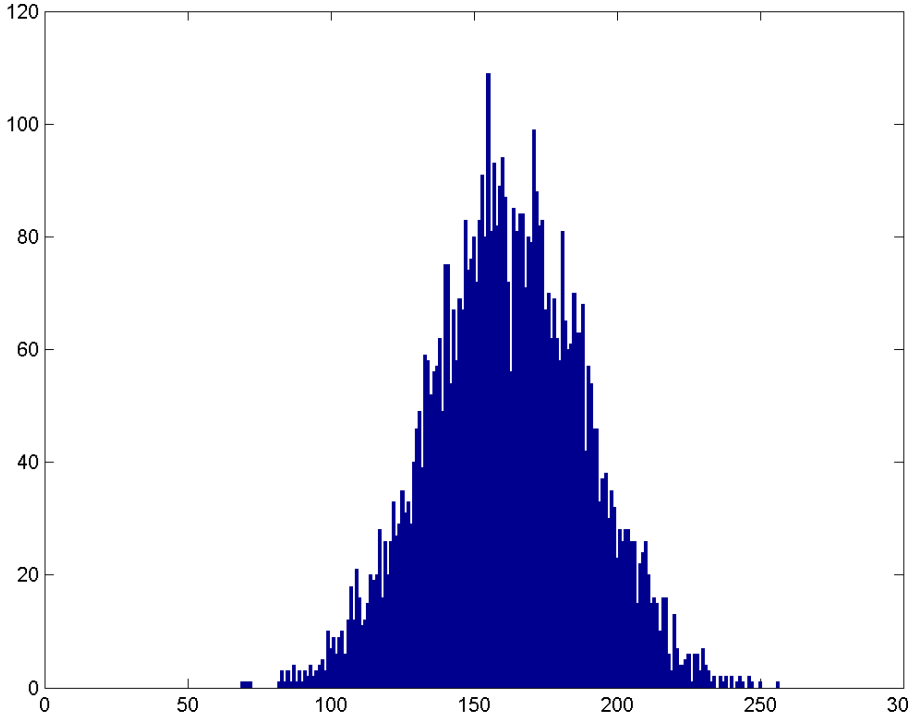


Homework

- 画出题3中图像的噪声的分布特性。

[B, c, r]=roipoly(f1);
（选取右上角的空白处）
>> [p, npix]=histroi(f1,c,r);
>> figure, bar(p, 1)







空间滤波

- 空间滤波基础
  - 线性空间滤波的原理
  - 空间相关和卷积
- 平滑空间滤波器
  - 线性空间滤波的原理
  - 均值滤波器和中值滤波器
- 锐化空间滤波器
  - 图像中的一阶和二阶导数
  - 拉普拉斯算子
  - 钝化掩蔽
- 组合使用空间增强方法